

“Hough” is pronounced “Huff.”

apply the distance transform to approximate distances from the region containing 1's to all other pixels in the image, using the masks:

$$\begin{array}{l}
 \text{(i)} \quad \begin{array}{ccc} & 1 & \\ 1 & 0 & 1 \\ & 1 & \end{array} \\
 \text{(ii)} \quad \begin{array}{ccc} & 1 & \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{array} \\
 \text{(iii)} \quad \begin{array}{ccc} & 4 & 3 & 4 \\ & 3 & 0 & 3 \\ 4 & 3 & 4 \end{array} \\
 \text{(iv)} \quad \begin{array}{ccccc} & & 11 & & 11 \\ 11 & 7 & 5 & 7 & 11 \\ & 5 & 0 & 5 & \\ 11 & 7 & 5 & 7 & 11 \\ & 11 & & 11 & \end{array}
 \end{array}$$

and applying any necessary scaling at the end.

- 28. Apply the distance transform and use it to find the skeleton in the images `circles2.png` and `nicework.png`.
- 29. Compare the result of the previous question with the results given by the Zhang-Suen and Guo-Hall methods. Which method produces the most visually appealing results? Which method seems fastest?

10 Mathematical Morphology

10.1 Introduction

Mathematical morphology, or *morphology* for short, is a branch of image processing that is particularly useful for analyzing shapes in images. We shall develop basic morphological tools for investigation of binary images, and then show how to extend these tools to grayscale images. MATLAB has many tools for binary morphology in the image processing toolbox; most of which can be used for grayscale morphology as well.

10.2 Basic Ideas

The theory of mathematical morphology can be developed in many different ways. We shall adopt one standard method which uses operations on sets of points. A very solid and detailed account can be found in Haralick and Shapiro [14].

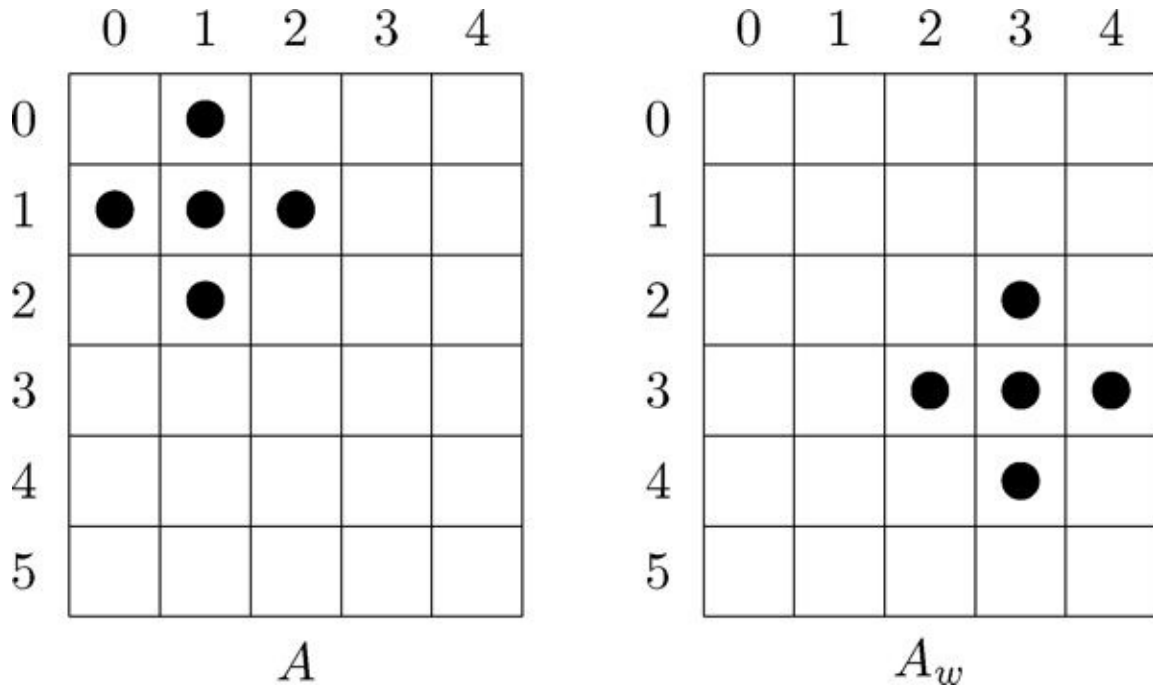
Translation

Suppose that A is a set of pixels in a binary image, and $w = (x, y)$ is a particular coordinate point. Then A_w is the set A “translated” in direction (x, y) . That is

$$A_x = \{(a, b) + (x, y) : (a, b) \in A\}.$$

For example, in Figure 10.1, A is the cross-shaped set and $w = (2, 2)$. The set A has been

Figure 10.1: Translation



shifted in the x and y directions by the values given in w . Note that here we are using matrix coordinates, rather than Cartesian coordinates, so that the origin is at the top left, x goes down and y goes across.

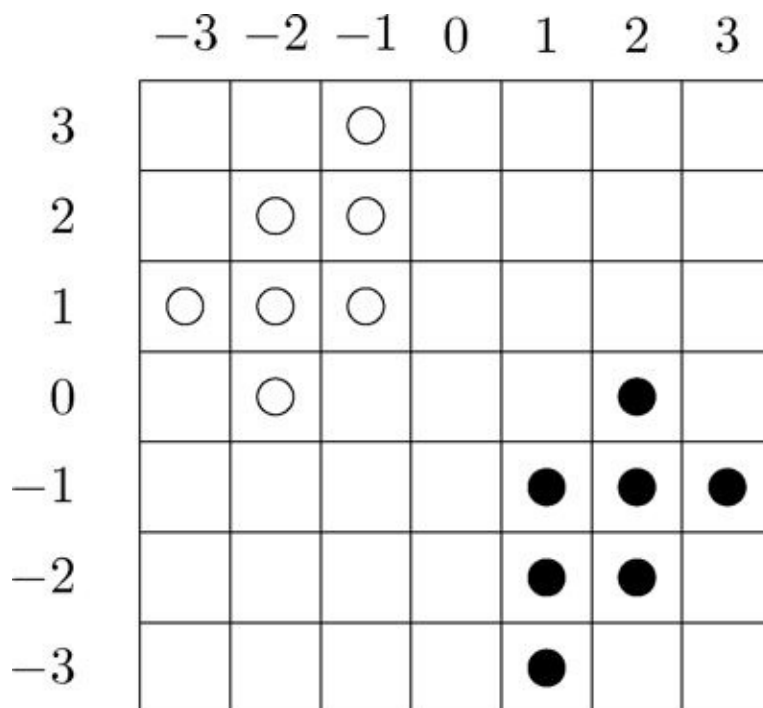
Reflection

If A is a set of pixels, then its *reflection*, denoted $\hat{A}$, is obtained by reflecting A in the origin:

$$\hat{A} = \{(-x, -y) : (x, y) \in A\}.$$

For example, in Figure 10.2, the open and closed circles form sets which are reflections of each other.

Figure 10.2: Reflection



10.3 Dilation and Erosion

These are the basic operations of morphology, in the sense that all other operations are built from a combination of these two.

Dilation

Suppose A and B are sets of pixels. Then the *dilation of A by B* , denoted $A \oplus B$, is defined as

$$A \oplus B = \bigcup_{x \in B} A_x.$$

What this means is that for every point $x \in B$, we translate A by those coordinates. Then we take the union of all these translations.

An equivalent definition is that

$$A \oplus B = \{(x, y) + (u, v) : (x, y) \in A, (u, v) \in B\}.$$

From this last definition, dilation is shown to be commutative; that is,

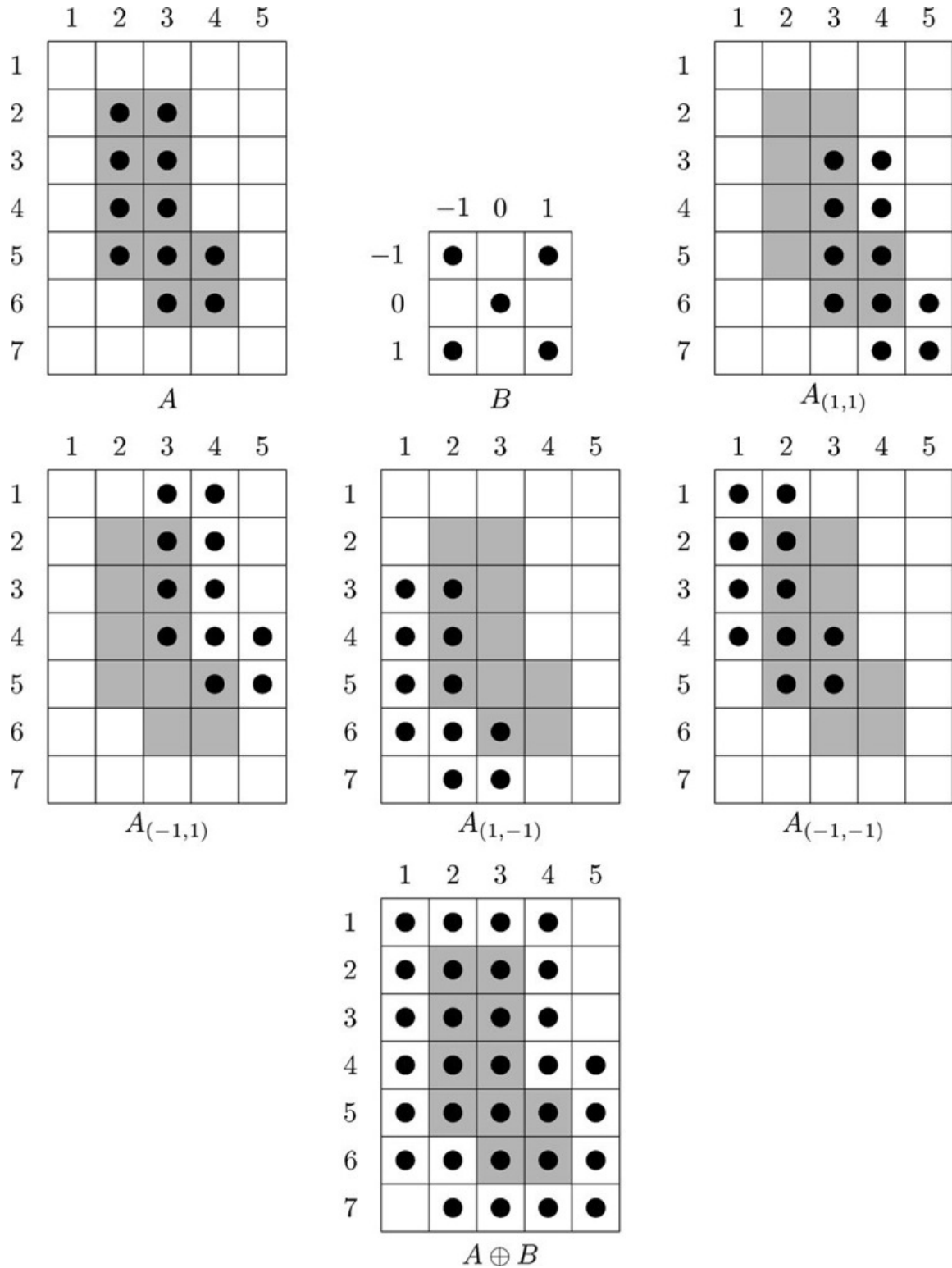
$$A \oplus B = B \oplus A.$$

An example of a dilation is given in Figure 10.3. In the translation diagrams, the gray squares show the original position of the object. Note that $A_{(0,0)}$ is of course just A itself. In this example, we have

$$B = \{(0, 0), (1, 1), (-1, 1), (1, -1), (-1, -1)\}$$

and these are the coordinates by which we translate A . In general, $A \oplus B$ can be obtained by replacing every point (x, y) in A with a copy of B , and placing the $(0, 0)$ point of B at (x, y) . Equivalently, we can replace every point (u, v) of B with a copy of A .

Figure 10.3: Dilation



Dilation is also known as *Minkowski addition*; see Haralick and Shapiro [14] for more information.

As you see in Figure 10.3, dilation has the effect of increasing the size of an object. However, it is not necessarily true that the original object A will lie within its dilation $A \oplus B$. Depending on the coordinates of B , $A \oplus B$ may end up quite a long way from A . Figure 10.4 gives an example of this: A is the same as in Figure 10.3; B has the same shape but a different position. In this figure, we have

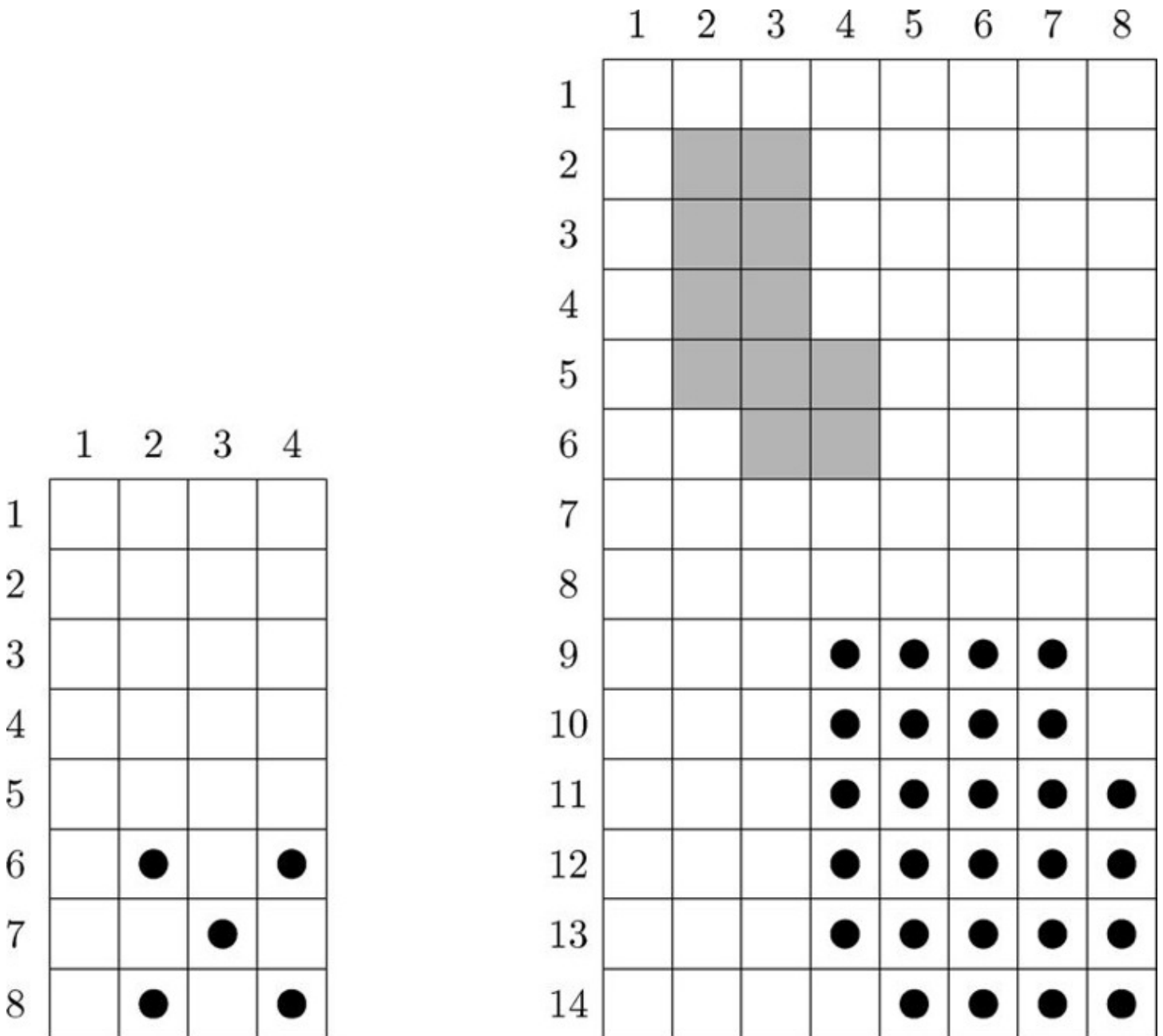
$$B = \{(7, 3), (6, 2), (6, 4), (8, 2), (8, 4)\}$$

so that

$$A \oplus B = A_{(7,3)} \cup A_{(6,2)} \cup A_{(6,4)} \cup A_{(8,2)} \cup A_{(8,4)}.$$

For dilation, we generally assume that A is the image being processed, and B is a small set of pixels. In this case, B is referred to as a *structuring element* or a *kernel*.

Figure 10.4: A dilation for which $A \not\subseteq A \oplus B$



Dilation in MATLAB or Octave can be performed with the command

```
>> imdilate(image, kernel)
```

MATLAB/Octave

and in Python with either of the commands `dilation` or `binary_dilation` from the `morphology` module of `skimage`. Both commands produce the same output for binary images, but `binary_dilation` is designed to be faster. To see an example of dilation, consider the commands:

```
>> t = imread('morph_text.png');
>> sq = ones(3,3);
>> td = imdilate(t,sq);
>> imshow(t)
>> figure,imshow(td)
```

MATLAB/Octave

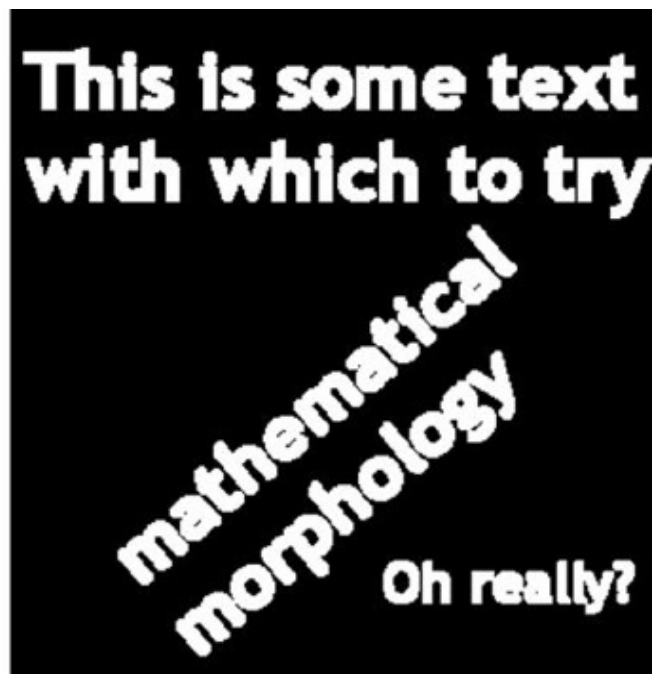
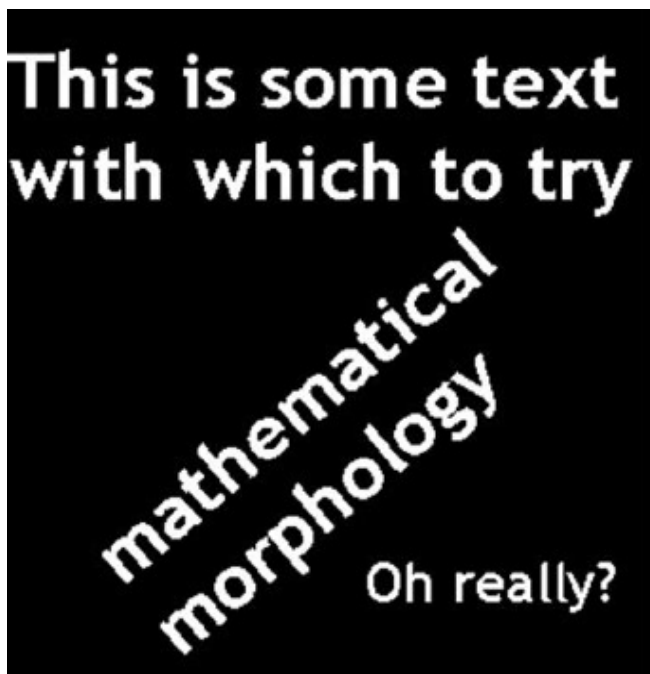
or

```
In : from skimage.morphology import binary_dilation as bwdilate
In : t = io.imread('morph_text.png')
In : sq = ones((3,3))
In : td = bwdilate(t,sq)
```

Python

The result is shown in Figure 10.5. Notice how the image has been “thickened.” This is really what dilation does; hence its name.

Figure 10.5: Dilation of a binary image



Erosion

Given sets A and B , the *erosion* of A by B , written $A \ominus B$, is defined as:

$$A \ominus B = \{w : B_w \subseteq A\}.$$

In other words, the erosion of A by B consists of all points $w = (x, y)$ for which B_w is in A . To perform an erosion, we can move B over A , and find all the places it will fit, and for each such place mark down the corresponding $(0, 0)$ point of B . The set of all such points will form the erosion.

An example of erosion is given in Figure 10.6.

Note that in the example, the erosion $A \ominus B$ was a subset of A . This is not necessarily the case; it depends on the position of the origin in B . If B contains the origin (as it did in Figure 10.6), then the erosion will be a subset of the original object.

Figure 10.7 shows an example where B does not contain the origin. In this figure, the open circles in the right-hand figure form the erosion.

Figure 10.6: Erosion with a cross-shaped structuring element

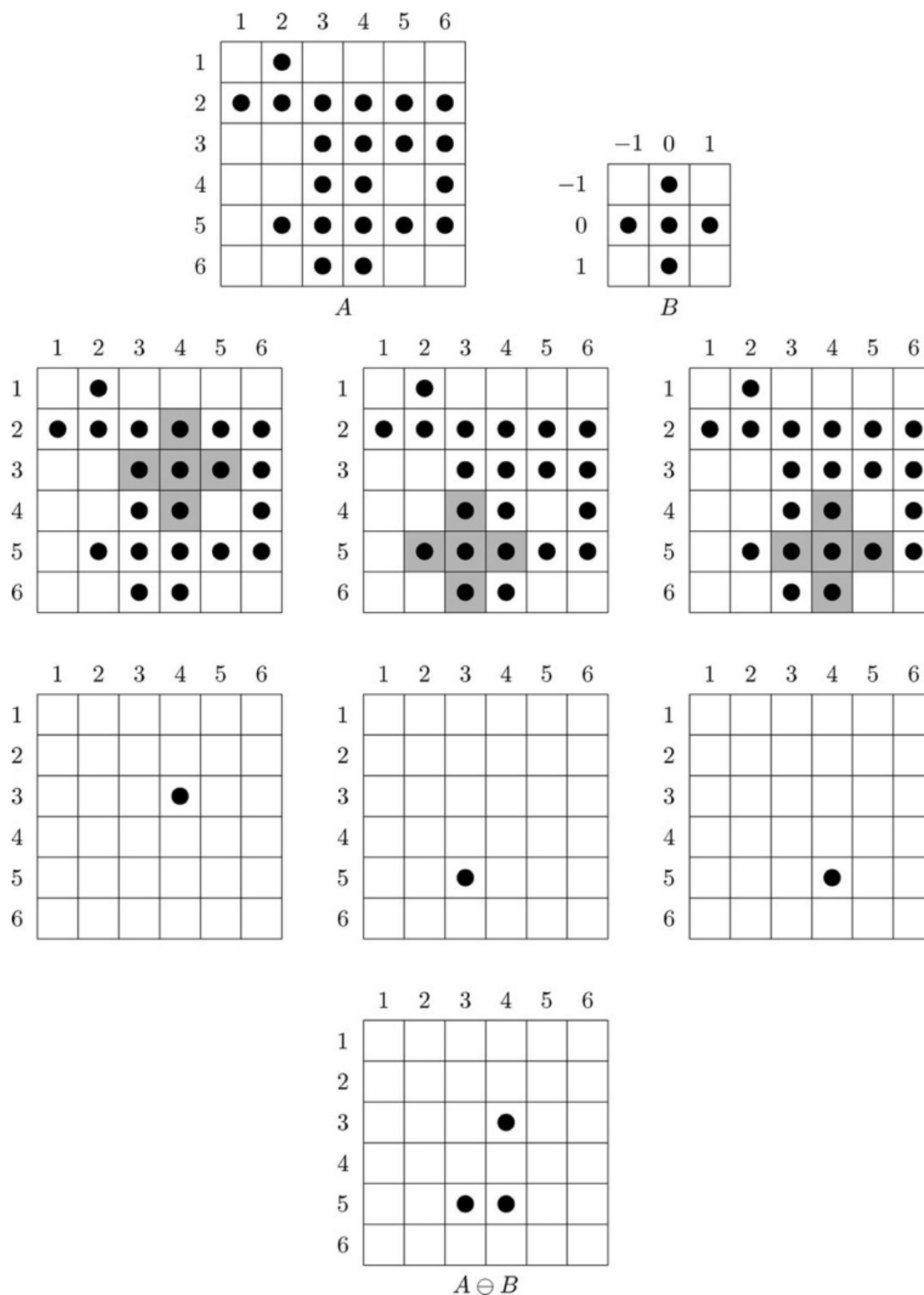
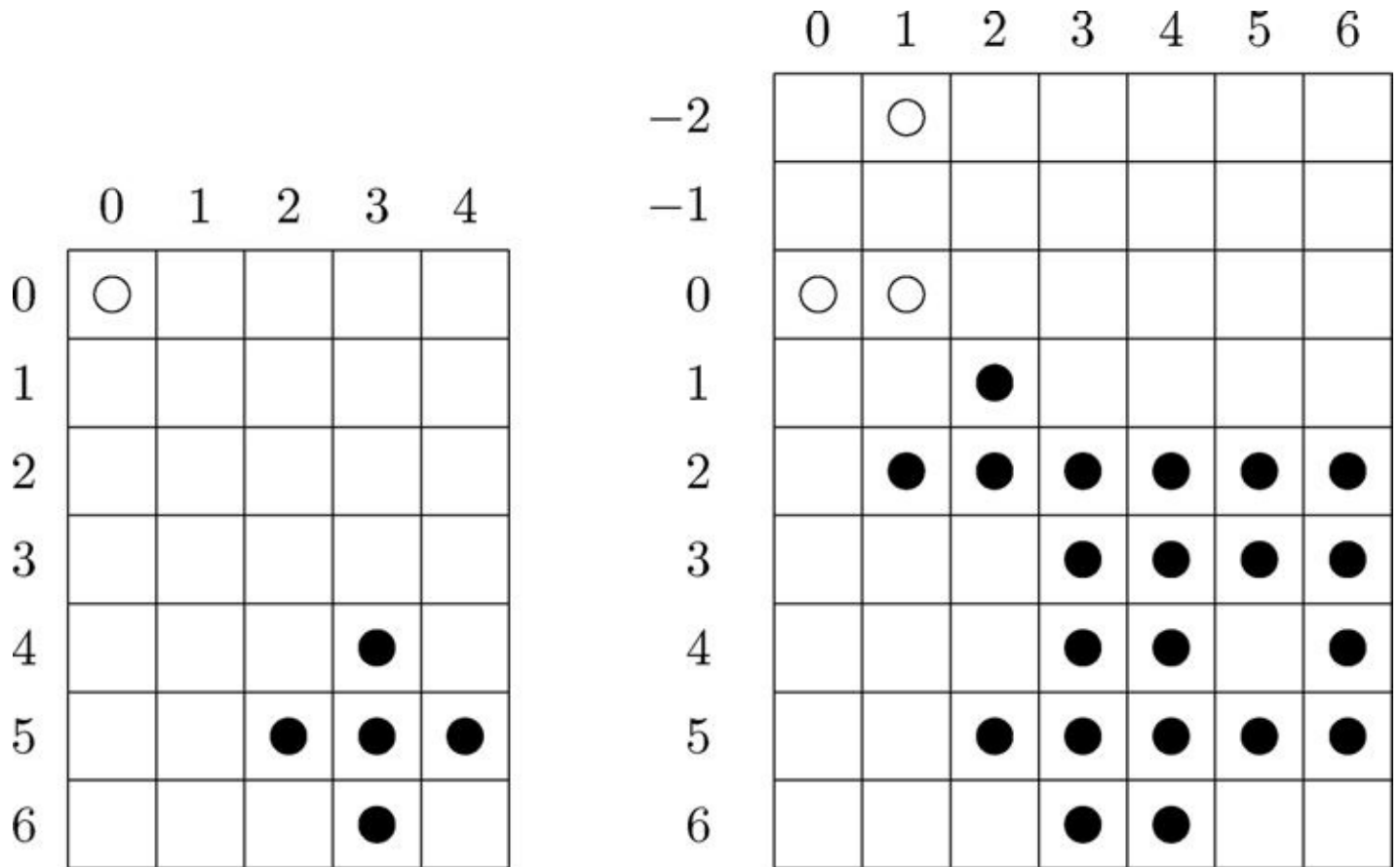


Figure 10.7: Erosion with a structuring element not containing the origin



Note that in Figure 10.7, the *shape* of the erosion is the same as that in Figure 10.6; however its *position* is different. Since the origin of B in Figure 10.7 is translated by $(-4, -3)$ from its position in Figure 10.6, we can assume that the erosion will be translated by the same amount. And if we compare Figures 10.6 and 10.7, we can see that the second erosion has indeed been shifted by $(-4, -3)$ from the first.

For erosion, as for dilation, we generally assume that A is the image being processed, and B is a small set of pixels: the structuring element or kernel.

Erosion is related to *Minkowski subtraction*: the Minkowski subtraction of B from A is defined as

$$A - B = \bigcap_{b \in B} A_b.$$

Erosion in MATLAB is performed with the command

```
>> imerode(image, kernel)
```

MATLAB/Octave

and in Python with either of the commands `erosion` or `binary_erosion`. We shall give an example using a different binary image:


```
>> r = imread('rings.png');
>> re = imerode(c,sq);
>> imshow(r)
>> figure,imshow(re)
```

MATLAB/Octave

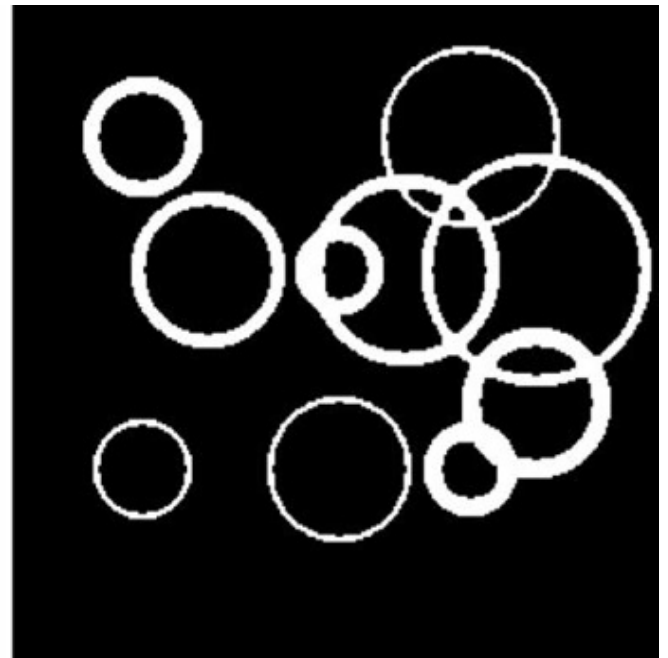
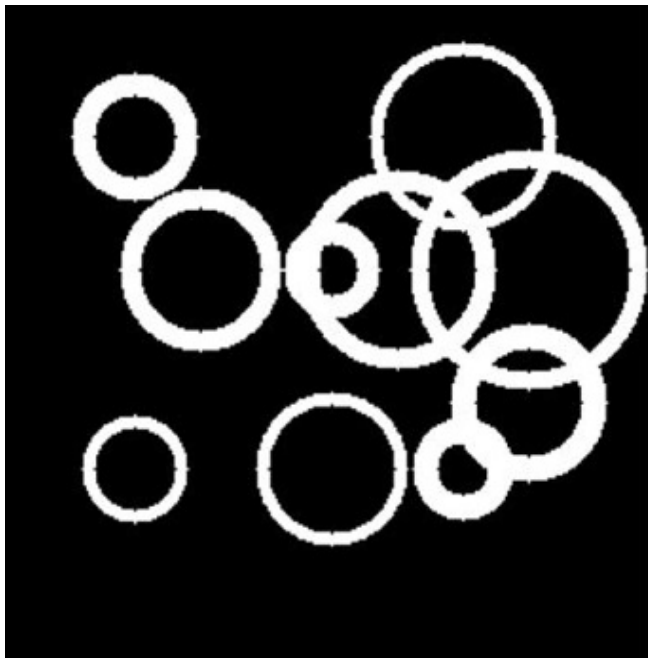
or

```
In : from skimage.morphology import binary_erosion as bwerode
In : r = io.imread('rings.png')
In : re = bwerode(r,sq)
In : io.imshow(r)
In : f = plt.figure(); f.show(io.imshow(re))
```

Python

The result is shown in Figure 10.8. Notice how the image has been “thinned.” This is the expected result of an erosion; hence its name. If we kept on eroding the image, we would end up with a completely black result.

Figure 10.8: Erosion of a binary image



Relationship between Erosion and Dilation

It can be shown that erosion and dilation are “inverses” of each other; more precisely, the complement of an erosion is equal to the dilation of the complement with the reflection of the structuring element. Thus:

$$\overline{A \ominus B} = \overline{A} \oplus \hat{B}.$$

A proof of this can be found in Haralick and Shapiro [14].

It can be similarly shown that the same relationship holds if erosion and dilation are interchanged; that

$$\overline{A \oplus B} = \overline{A} \oplus \hat{B}.$$

We can demonstrate the truth of these using MATLAB commands; all we need to know is that the complement of a binary image

b

is obtained using

```
>> ~b
```

MATLAB/Octave

and that given two images a and b, their equality can be determined in MATLAB and Octave with

```
>> isequal(a,b)
```

MATLAB/Octave

and in Python with

```
In : np.array_equal(a,b)
```

Python

To demonstrate the equality

$$\overline{A \oplus B} = \overline{A} \oplus \hat{B}.$$

pick a binary image, say the text image, and a structuring element. Then the left-hand side of this equation is produced with

```
>> lhs=~imerode(t,sq);
```

MATLAB/Octave

and the right-hand side with

```
>> rhs=imdilate(~t,sq);
```

MATLAB/Octave

Finally, the command

```
>> isequal(lhs,rhs)
```

MATLAB/Octave

should return 1, for true. In Python, to make such commands work, the starting image must have values 0,1 only:

```
>> c = io.imread('morph_text.png').astype('bool')*1
```

Python

Then testing the equations is pretty much the same, but since the result of a morphological operation is an array of zero and ones, a logical complement cannot be used:

```
In : lhs = 1-bwerode(c,sq)
In : rhs = bwdilate(1-c,sq)
In : np.array_equal(rhs,lhs)
```

Python

An Application: Boundary Detection

If A is an image, and B a small structuring element consisting of point symmetrically places about the origin, then we can define the boundary of A by any of the following methods:

- (i) $A - (A \ominus B)$ “internal boundary”
- (ii) $(A \ominus B) - A$ “external boundary”
- (iii) $(A \ominus B) - (A \ominus B)$ “morphological gradient”

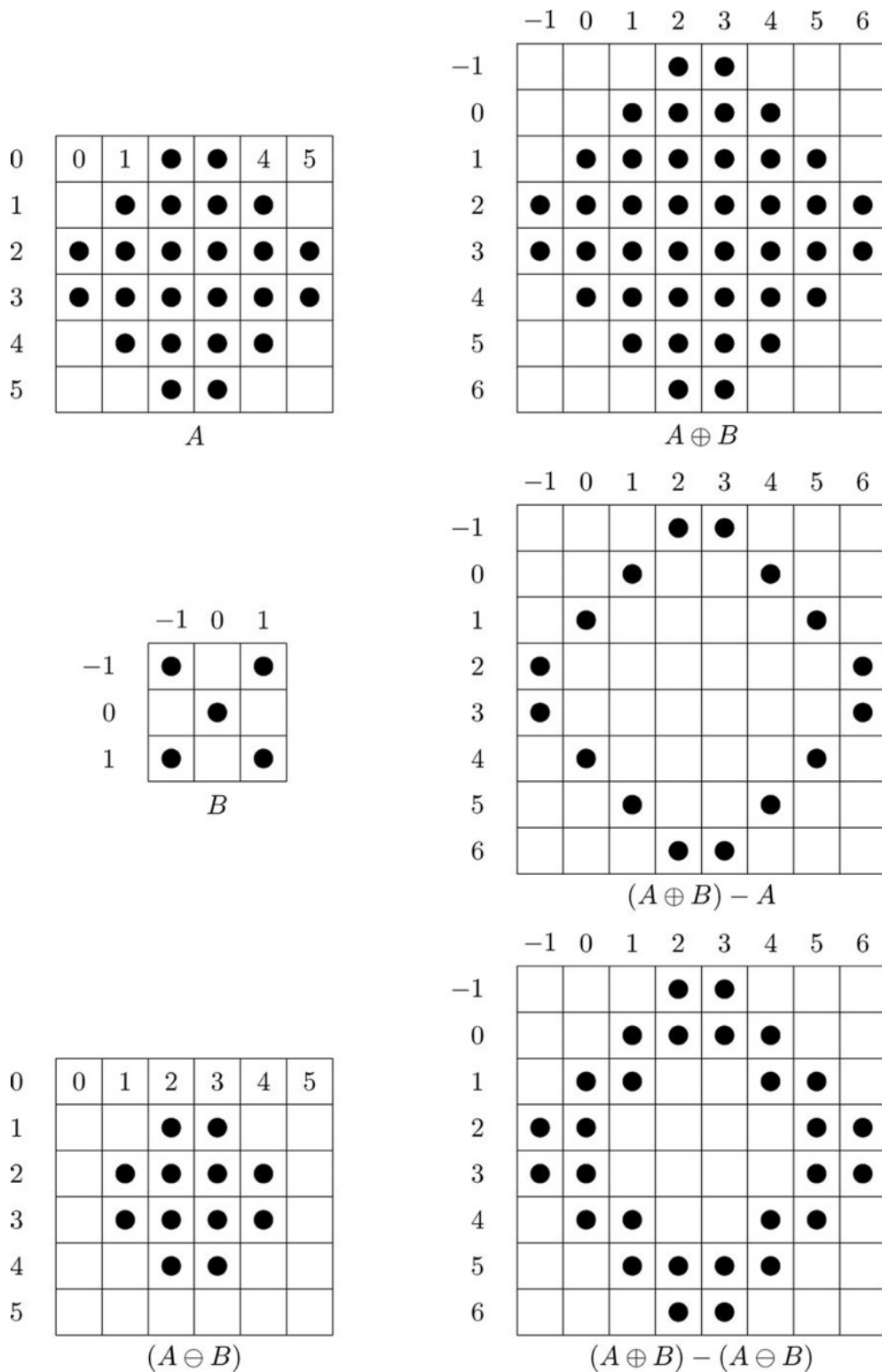
In each definition, the minus refers to set difference. For some examples, see Figure 10.9. Note that the internal boundary consists of those pixels in A that are at its edge; the external boundary consists of pixels outside A and which are just next to it, and the morphological gradient is a combination of both the internal and external boundaries.

To see some examples, choose the image `pinenuts.png`, and threshold it to obtain a binary image:

```
>> n = imread('pinenuts.png');
>> p = ~(p>130 & p<165);
```

MATLAB/Octave

Figure 10.9: Boundaries



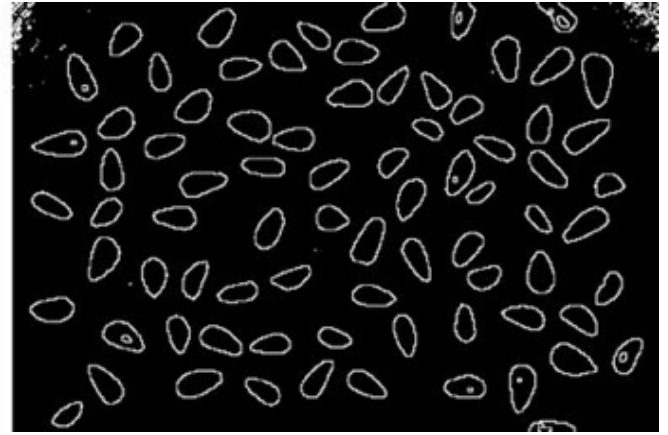
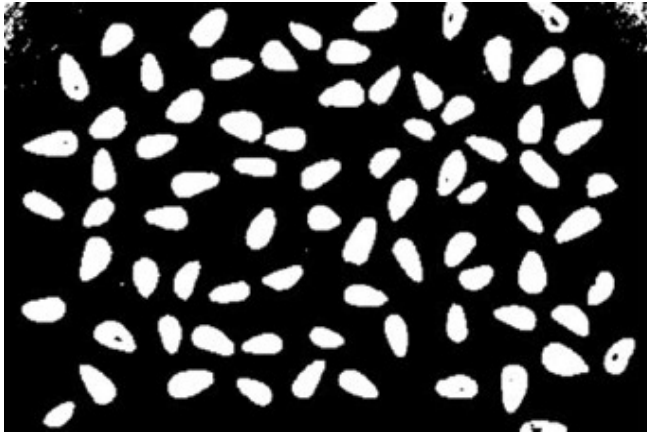
Then the internal boundary is obtained with:

```
>> pe = imerode(p,sq);
>> p_int = p & ~re;
>> imshow(p),figure,imshow(p_int)
```

MATLAB/Octave

The result is shown in Figure 10.10. The above commands can be implemented in Python

Figure 10.10: “Internal boundary” of a binary image



as

```
In : n = io.imread('pinenuts.png')
In : p = ~(n>130) & (n<165)
In : pe = bwerode(p,sq)
In : p_int = p-pe
In : io.imshow(p)
In : f = plt.figure(); f.show(io.imshow(p-pe))
```

Python

The external boundary and morphological gradients can be obtained similarly:

```
>> pd = imdilate(p,sq);
>> p_ext = pd & ~p;
>> p_grad = pd & ~pe;
>> imshow(p_ext), figure,imshow(p_grad)
```

MATLAB/Octave

or with

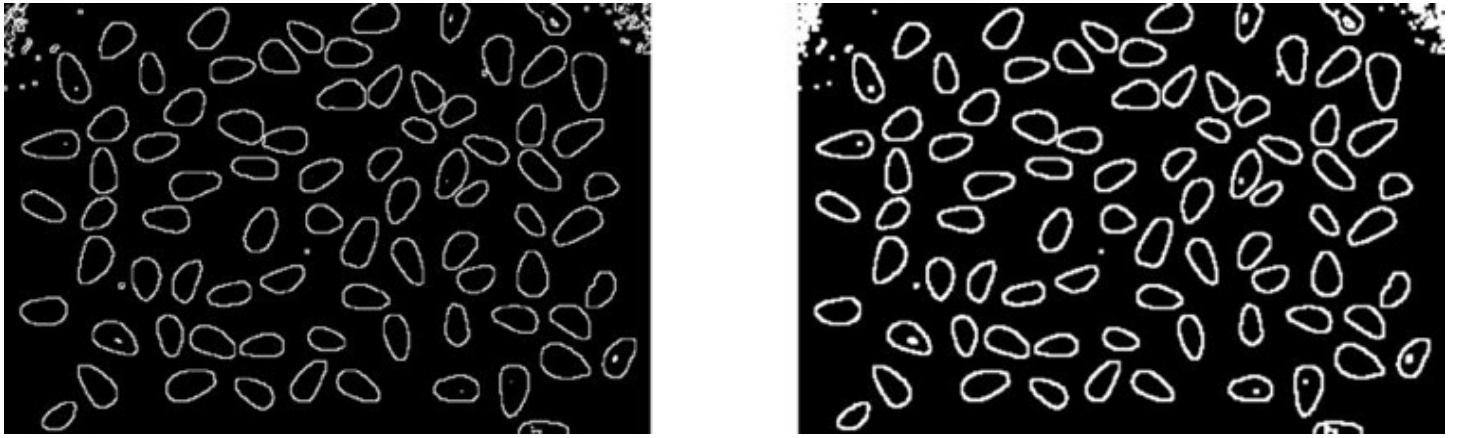
```
In : pd = bwdilate(p,sq)
In : p_exp = pd-p
In : p_grad = pd-pe
In : io.imshow(p_ext)
In : f = plt.figure(); f.show(io.imshow(p_grad))
```

Python

The results are shown in Figure 10.11.

Note that the external boundaries are larger than the internal boundaries. This is because the internal boundaries show the outer edge of the image components whereas the external boundaries show the pixels just outside the components. The morphological gradient is thicker than either, and is in fact the union of both.

Figure 10.11: “External boundary” and the morphological gradient of a binary image



10.4 Opening and Closing

These operations may be considered as “second level” operations in that they build on the basic operations of dilation and erosion. They are also, as we shall see, better behaved mathematically.

Opening

Given A and a structuring element B , the *opening of A by B* , denoted $A \circ B$, is defined as:

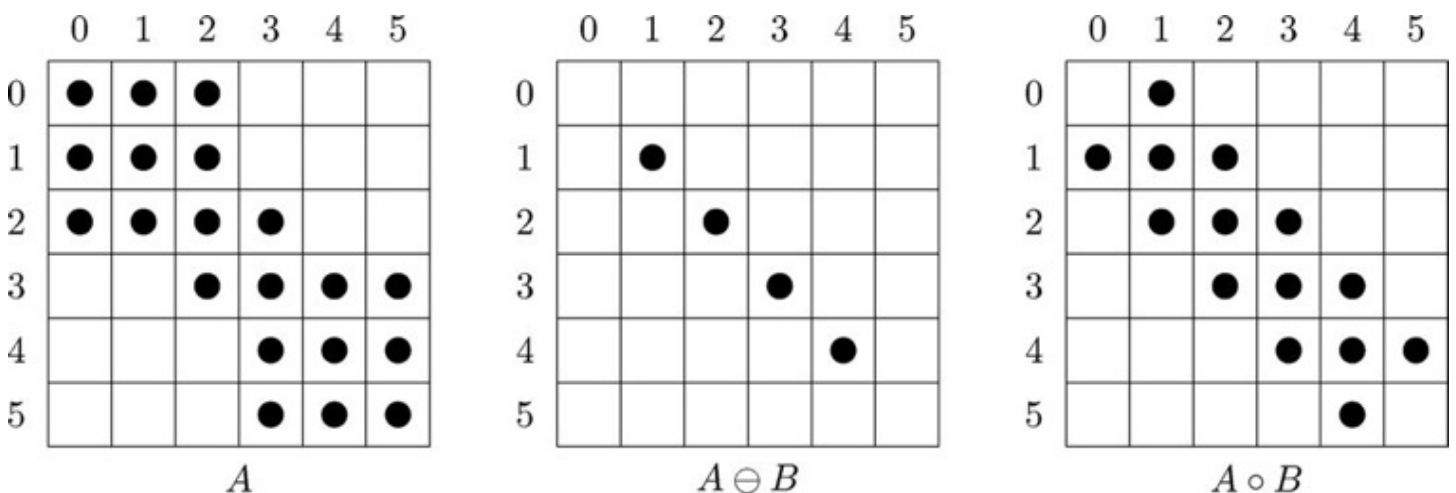
$$A \circ B = (A \ominus B) \oplus B.$$

So, an opening consists of an erosion followed by a dilation. An equivalent definition is

$$A \circ B = \cup \{B_w : B_w \subseteq A\}.$$

That is, $A \circ B$ is the union of all translations of B that fit inside A . Note the difference with erosion: the erosion consists only of the (0, 0) point of B for those translations that fit inside A ; the opening consists of all of B . An example of opening is given in Figure 10.12.

Figure 10.12: Opening



The opening operation satisfies the following properties:

1. $(A \circ B) \subseteq A$. Note that this is not the case with erosion; as we have seen, an erosion may not necessarily be a subset.

- 2. $(A \circ B) \circ B = A \circ B$. That is, an opening can never be done more than once. This property is called *idempotence*. Again, this is not the case with erosion; you can keep on applying a sequence of erosions to an image until nothing is left.
- 3. If $A \subseteq C$, then $(A \circ B) \subseteq (C \circ B)$.
- 4. Opening tends to “smooth” an image, to break narrow joins, and to remove thin protrusions.

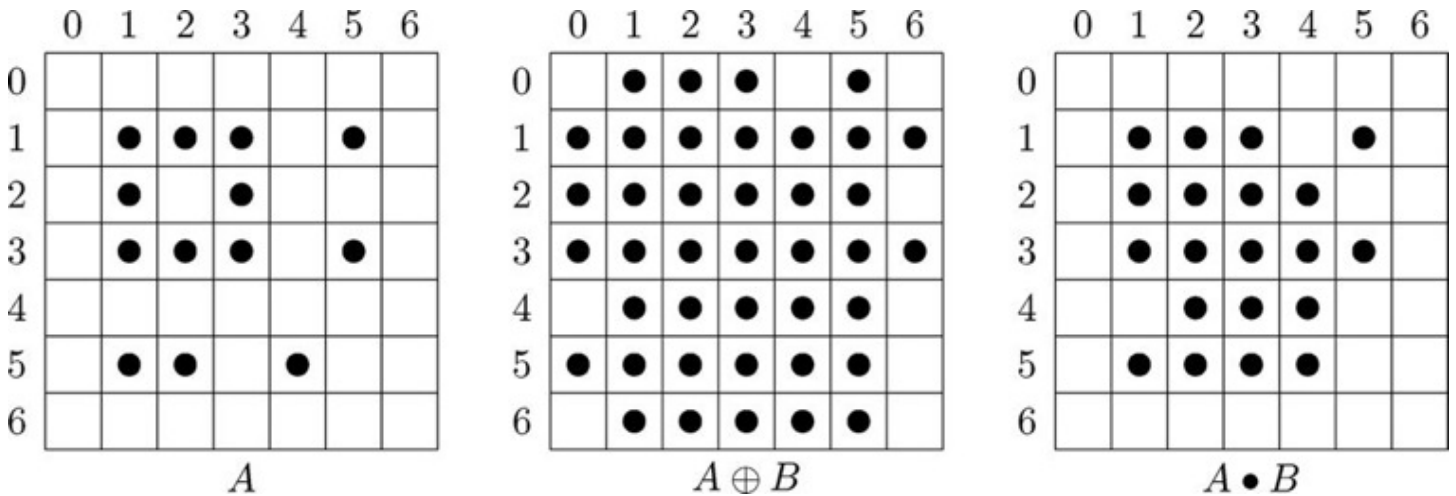
Closing

Analogous to opening we can define *closing*, which may be considered a dilation followed by an erosion, and is denoted $A \bullet B$:

$$A \bullet B = (A \oplus B) \ominus B.$$

Another definition of closing is that $x \in A \bullet B$ if *all* translations B_w that contain x have non-empty intersections with A . An example of closing is given in Figure 10.13.

Figure 10.13: Closing



The closing operation satisfies the following properties:

1. $A \subseteq (A \bullet B)$.
2. $(A \bullet B) \bullet B = A \bullet B$; that is, closing, like opening, is idempotent.
3. If $A \subseteq C$, then $(A \bullet B) \subseteq (C \bullet B)$.
4. Closing also tends to smooth an image, but it fuses narrow breaks and thin gulfs, and eliminates small holes.

Opening and closing are implemented by the `imopen` and `imclose` functions, respectively. We can see the effects on a simple image using the square and cross-shaped structuring elements.

```
>> cr=[0 1 0;1 1 1;0 1 0];
>> test=zeros(10,10);test(2:6,2:4)=1;test(3:5,6:9)=1;test(8:9,4:8)=1;test
(4,5)=1
```

MATLAB/Octave

or with

```
In : cr = np.array([[0,1,0],[1,1,1],[0,1,0]])
In : test = np.zeros((10,10))
In : test[1:6,1:4] = 1; test[2:5,5:9] = 1; test[7:9,3:8] = 1; test[3,5] =
1
```

Python

It looks like this:

```
0 0 0 0 0 0 0 0 0 0
0 1 1 1 0 0 0 0 0 0
0 1 1 1 0 1 1 1 1 0
0 1 1 1 1 1 1 1 1 0
0 1 1 1 0 1 1 1 1 0
0 1 1 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 1 1 1 1 1 0 0
0 0 0 1 1 1 1 1 0 0
0 0 0 0 0 0 0 0 0 0
```

Openings and closings can be done very easily:

```
>> test_open = imopen(test,cr);
>> test_close = imclose(test,cr)
```

MATLAB/Octave

or with Python, making sure to import the relevant commands first:

```
In : from skimage.morphology import closing, opening, binary_closing as
      bwclose, binary_opening as bwopen
In : test_open = bwopen(test,cr);
In : test_close = bwclose(test,cr)
```

Python

The outputs are shown in Figure 10.14. Opening and closing with the square structuring

Figure 10.14: Open and closing with the cross-shaped structuring element

```

0 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 0 0 0
0 1 1 1 0 1 1 1 0 0
0 1 1 1 1 1 1 1 1 0
0 1 1 1 0 1 1 1 0 0
0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0

```

(a) Opening

```

0 0 1 0 0 0 0 0 0 0
0 1 1 1 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 0
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 0
0 1 1 1 1 1 1 1 0 0
0 0 1 1 1 1 1 0 0 0
0 0 0 1 1 1 1 1 0 0
0 0 0 1 1 1 1 1 0 0
0 0 0 0 1 1 1 0 0 0

```

(b) Closing

element are shown in Figure 10.15. With closing, the image is now fully “joined up.” We can obtain a smeared effect with the text image, using a diagonal structuring element:

Figure 10.15: Opening and closing with the square structuring element

```

0 0 0 0 0 0 0 0 0 0
0 1 1 1 0 0 0 0 0 0
0 1 1 1 0 1 1 1 1 0
0 1 1 1 0 1 1 1 1 0
0 1 1 1 0 1 1 1 1 0
0 1 1 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0

```

(a) Opening

```

1 1 1 1 0 0 0 0 0 0
1 1 1 1 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 0 0
0 0 0 1 1 1 1 1 0 0
0 0 0 1 1 1 1 1 0 0
0 0 0 1 1 1 1 1 0 0
0 0 0 1 1 1 1 1 0 0

```

(b) Closing

```

>> k = [0 0 0 0 1 1;0 0 1 1 0 0;1 1 0 0 0 0]
k =

```

```

0 0 0 0 1 1
0 0 1 1 0 0
1 1 0 0 0 0

```

```

>> tc = imclose(t,k);
>> imshow(tc)

```

The result is shown in Figure 10.16.

Figure 10.16: An example of closing



An Application: Noise Removal

Suppose A is a binary image corrupted by impulse noise—some of the black pixels are white and some of the white pixels are black. An example is given in Figure 10.17. Then $A \ominus B$ will remove the single black pixels, but will enlarge the holes. We can fill the holes by dilating twice:

$$((A \ominus B) \oplus B) \oplus B.$$

The first dilation returns the holes to their original size; the second dilation removes them. But this will enlarge the objects in the image. To reduce them to their correct size, perform a final erosion:

$$(((A \ominus B) \oplus B) \oplus B) \ominus B.$$

The inner two operations constitute an opening; the outer two operations a closing. Thus, this noise removal method is in fact an opening followed by a closing:

$$(A \circ B) \bullet B.$$

This is called *morphological filtering*.

Suppose we take an image and apply 10% shot noise to it. This can be done in MATLAB or Octave by:

```
>> c = imread('circles.png');  
>> x = rand(size(c));  
>> c(find(x>0.95)) = 0;  
>> c(find(x<0.05)) = 1;  
>> imshow(c)
```

and in Python using the function from the module of numpy:

```
In : c = io.imread('circles.png').astype('bool')*1
In : x = np.random.random_sample(c.shape)
In : c[np.nonzero(x>0.95)]=0
In : c[np.nonzero(x<=0.05)]=1
```

Python

The result is shown as Figure 10.17(a). The filtering process can be implemented with

```
>> cf1 = imclose(imopen(c,sq),sq);
>> figure, imshow(cf1)
>> cf2 = imclose(imopen(c,cr),cr);
>> figure, imshow(cf2)
```

MATLAB/Octave

and the same in Python, with `imclose` and `imopen` replaced with our aliases `bwclose` and `bwopen`. The results are shown as Figures 10.17(b) and (c). The results are rather “blocky”; although less so with the cross-shaped structuring element.

Relationship between Opening and Closing

Opening and closing share a relationship very similar to that of erosion and dilation: the complement of an opening is equal to the closing of a complement, and the complement of a closing is equal to the opening of a complement. Specifically:

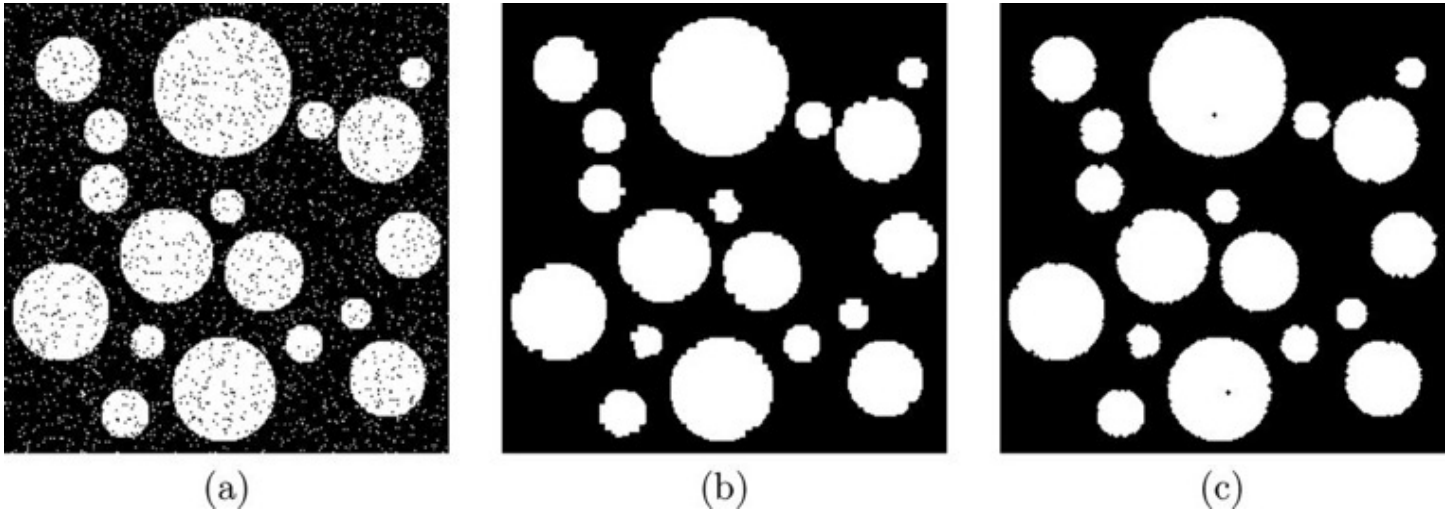
$$\overline{A \bullet B} = \overline{A} \circ \hat{B}$$

and

$$\overline{A \circ B} = \overline{A} \bullet \hat{B}.$$

Again, see Haralick and Shapiro [14] for a formal proof.

Figure 10.17: A noisy binary image and results after morphological filtering with different structuring elements.

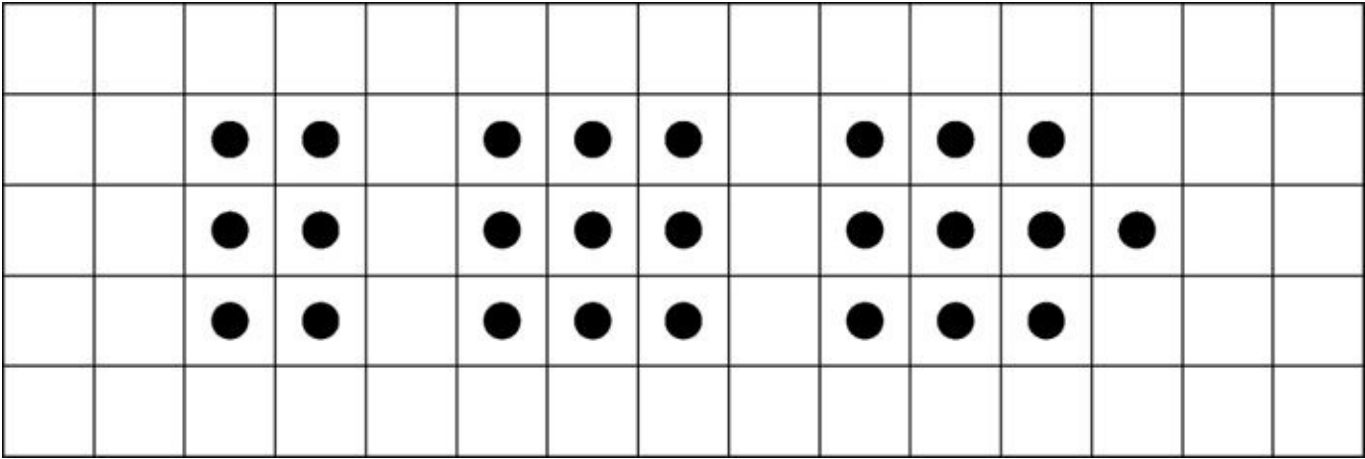


10.5 The Hit-or-Miss Transform

This is a powerful method for finding shapes in images. As with all other morphological algorithms, it can be defined entirely in terms of dilation and erosion; in this case, erosion only.

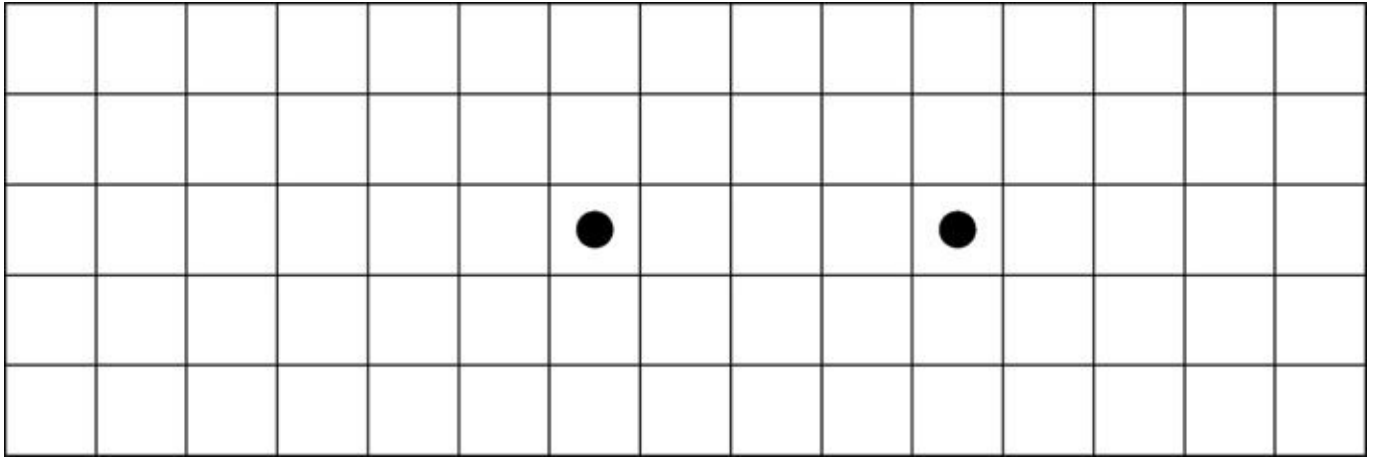
Suppose we wish to locate 3×3 square shapes, such as is in the center of the image *A* in Figure 10.18.

Figure 10.18: An image *A* containing a shape to be found



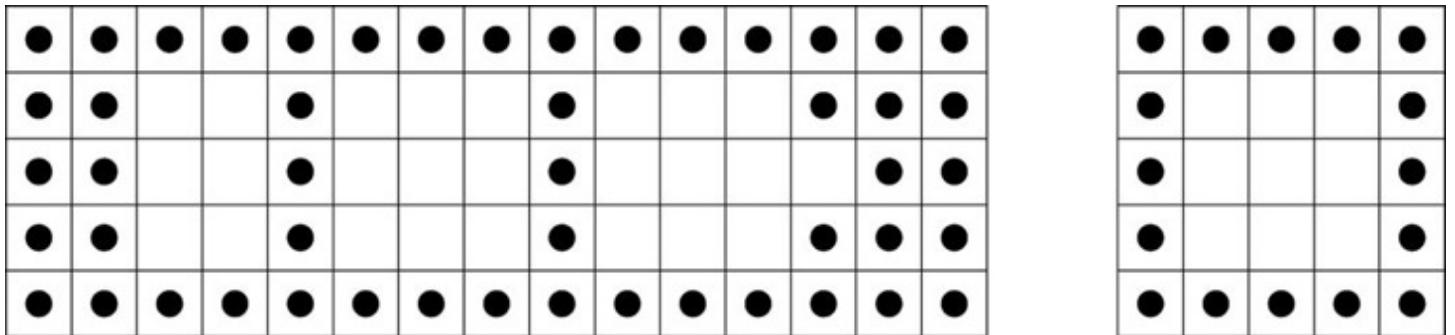
If we performed an erosion $A \ominus B$ with *B* being the square structuring element, we would obtain the result given in Figure 10.19.

Figure 10.19: The erosion $A \ominus B$



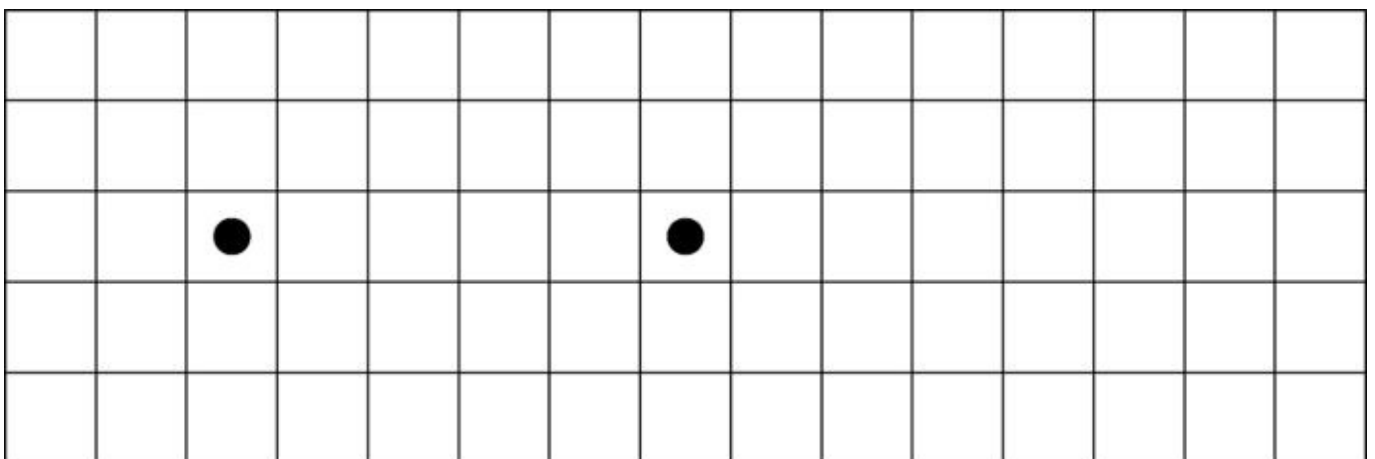
The result contains two pixels, as there are exactly two places in A where B will fit. Now suppose we also erode the complement of A with a structuring element C , which fits exactly around the 3×3 square; $\bar{A}$ and C are shown in Figure 10.20. (We assume that $(0, 0)$ is at the center of C .)

Figure 10.20: The complement $\bar{A}$ and the second structuring element



If we now perform the erosion $\bar{A} \ominus C$, we would obtain the result shown in Figure 10.21.

Figure 10.21: The erosion $\bar{A} \ominus C$



The intersection of the two erosion operations would produce just one pixel at the position of the center of the 3×3 square in A , which is just what we want. If A had contained more than one square, the final result would have been single pixels at the positions of the centers of each. This combination of erosions forms the hit-or-miss transform.

In general, if we are looking for a particular shape in an image, we design two structuring elements: B_1 , which is the same shape, and B_2 , which fits around the shape. We then write $B = (B_1, B_2)$ and

$$A \circledast B = (A \ominus B_1) \cap (\overline{A} \ominus B_2)$$

for the hit-or-miss transform.

As an example, we shall attempt to find the dot at the bottom of the question mark in the `morph_text.png` image. This is in fact a 4×4 square with missing corners. The two structuring elements then will be defined in MATLAB/Octave as

```
>> b1 = [0 1 1 0;1 1 1 1;1 1 1 1;0 1 1 0]
>> b2 = ones(6); b2(2:5,2:5) = ~b1
>> tb1 = imerode(t,b1);
>> tb2 = imerode(~t,b2);
>> hit_or_miss = tb1 & tb2;
>> [x,y] = find(hit_or_miss==1)
```

MATLAB/Octave

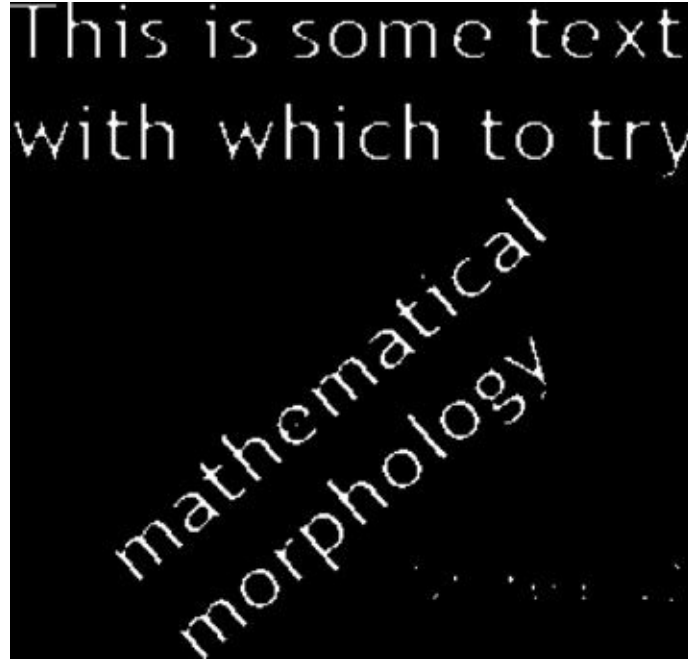
or in Python with

```
In : b1 = np.array([[0,1,1,0],[1,1,1,1],[1,1,1,1],[0,1,1,0]])
In : b2 = ones((6,6)); b2[1:5,1:5] = 1-b1
In : tb1 = bwerode(t,b1);
In : tb2 = bwerode(1-t,b2);
In : np.where((tb1 & tb2)==1)
```

Python

and this returns a coordinate of (281, 296) in MATLAB/Octave and (280, 295) in Python, which is right in the middle of the dot. Note that eroding the text by `b1` alone is not sufficient, as there are many places in the images where `b1` can fit. This can be seen by viewing the image `tb1`, which is given in Figure 10.22.

Figure 10.22: Text eroded by a dot-shaped structuring element



10.6 Some Morphological Algorithms

In this section we shall investigate some simple algorithms that use some of the morphological techniques we have discussed in previous sections.

Region Filling

Suppose in an image we have a region bounded by an 8-connected boundary, as shown in Figure 10.23.

Given a pixel p within the region, we wish to fill up the entire region. To do this, we start with p , and dilate as many times as necessary with the cross-shaped structuring element B (as used in Figure 10.6), each time taking an intersection with $\bar{A}$ before continuing. We thus create a sequence of sets:

$$\{p\} = X_0, X_1, X_2, \dots, X_k = X_{k+1}$$

for which

$$X_n = (X_{n-1} \oplus B) \cap \bar{A}.$$

Finally, $X_k \cup A$ is the filled region. Figure 10.24 shows how this is done.

Figure 10.23: An 8-connected boundary of a region to be filled

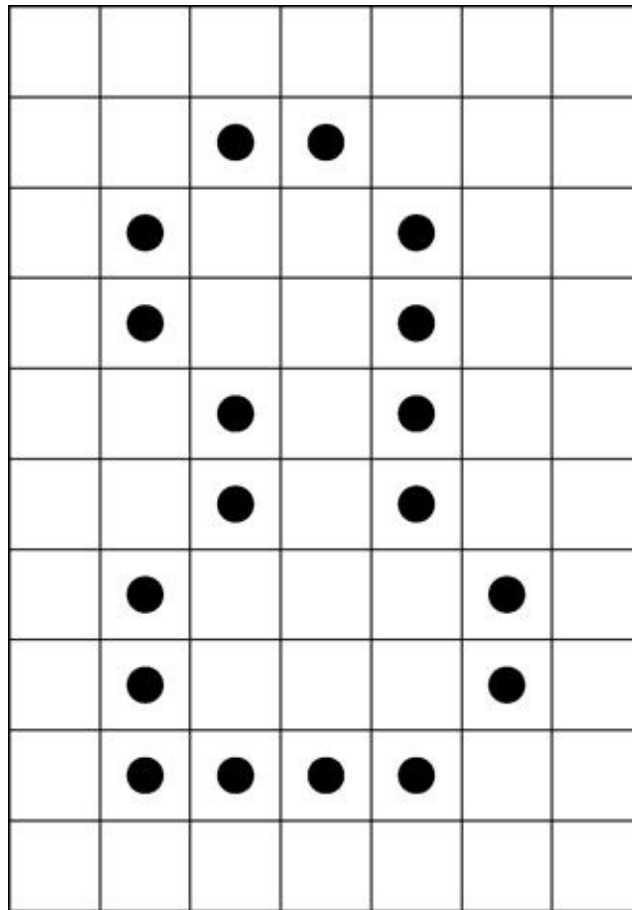
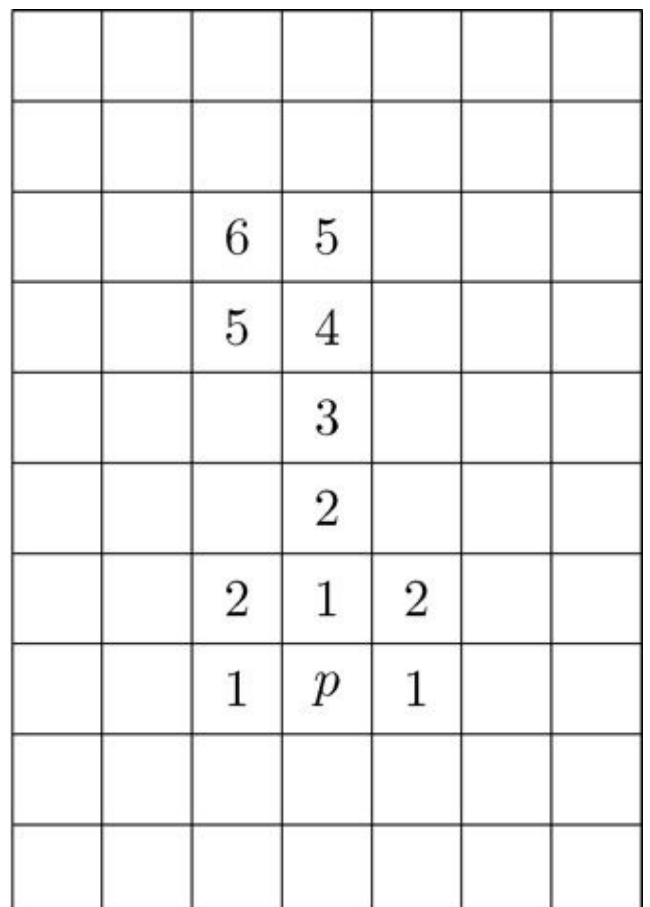
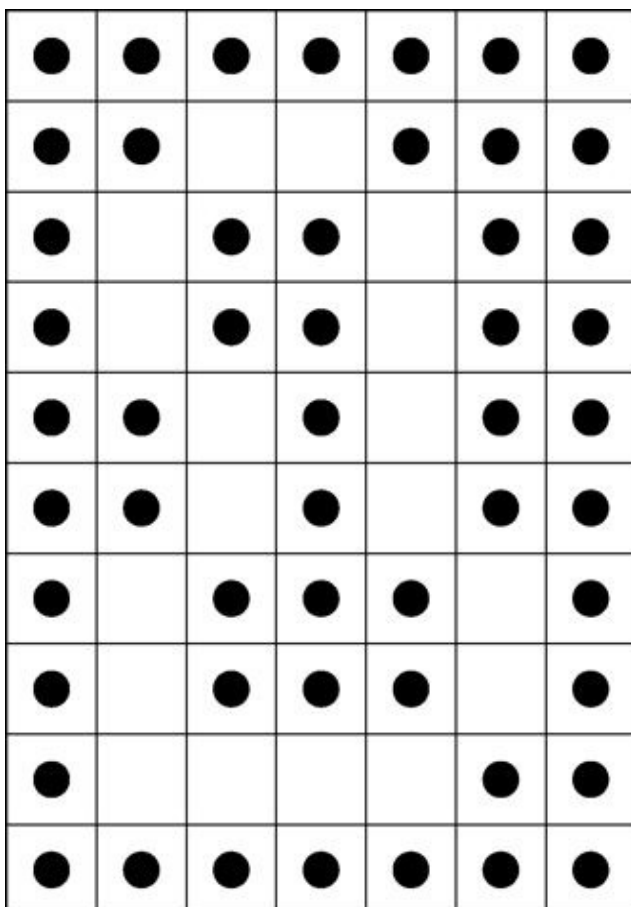


Figure 10.24: The process of filling the region



In the right-hand grid, we have

$$X_0 = \{p\}, X_1 = \{p, 1\}, X_2 = \{p, 1, 2\}, \dots$$

Note that the use of the cross-shaped structuring element means that we never cross the boundary.

Connected Components

We use a very similar algorithm to fill a connected component; we use the cross-shaped structuring element for 4-connected components, and the square structuring element for 8-connected components. Starting with a pixel p , we fill up the rest of the component by creating a sequence of sets

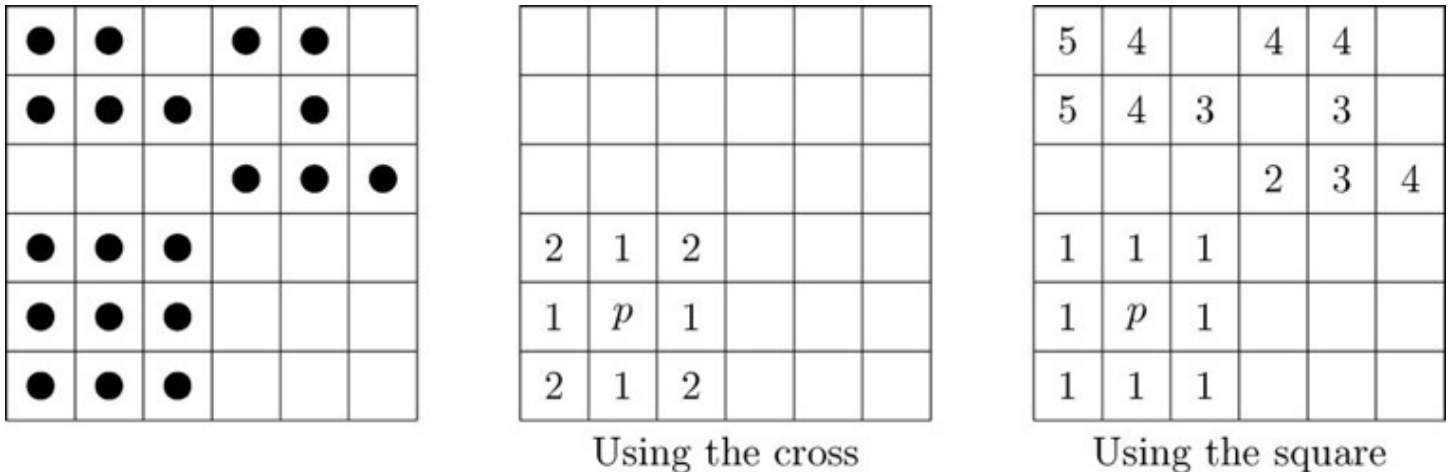
$$X_0 = \{p\}, X_1, X_2, \dots$$

such that

$$X_n = (X_{n-1} \oplus B) \cap A$$

until $X_k = X_{k-1}$. Figure 10.25 shows an example.

Figure 10.25: Filling connected components



In each case, we are starting in the center of the square in the lower left. As this square is itself a 4-connected component, the cross structuring element cannot go beyond it.

MATLAB and Octave implement this algorithm with the `bwlabel` function; Python with the `label` function in the `measure` module of `skimage`. Suppose X is the image shown in Figure 10.25. Then in MATLAB or Octave:

```
>> bwlabel(X,4)
>> bwlabel(X,8)
```

MATLAB/Octave

will show the 4-connected or 8-connected components; using the cross or square structuring elements, respectively. In Python:

```
In : from skimage.measure import label
In : bwlabel(X,4,background=0)+1
In : bwlabel(X,8,background=0)+1
```

Python

The background parameter tells Python which pixel values to treat as background; these are assigned the value -1, all foreground components are labeled starting with the index value 0. Adding 1 puts the background back to zero, and labels starting at 1.

MATLAB and Octave differ from Python in terms of the direction of scanning, as shown in Figure 10.26.

Figure 10.26: Labeling connected components

```
[[1 1 0 2 2 0]
 [1 1 1 0 2 0]
 [0 0 0 2 2 2]
 [3 3 3 0 0 0]
 [3 3 3 0 0 0]
 [3 3 3 0 0 0]]
```

(a) Python 4-components

```
1 1 0 3 3 0
1 1 1 0 3 0
0 0 0 3 3 3
2 2 2 0 0 0
2 2 2 0 0 0
2 2 2 0 0 0
```

(b) MATLAB/Octave 4-component

```
1 1 0 1 1 0
1 1 1 0 1 0
0 0 0 1 1 1
1 1 1 0 0 0
1 1 1 0 0 0
1 1 1 0 0 0
```

(c) 8-components

Filling holes can be done with the `bwfill` function in MATLAB/Octave, or the function `binary_fill_holes` from the `ndimage` module of `scipy`:

```
>> n = imread('nicework.png');
>> nb = n & ~imerode(n,sq);
>> imshow(nb)
>> nf = bwfill(nb,[74,52],sq);
>> figure,imshow(nf)
```

MATLAB/Octave

Python's command by default fills all holes by dilating and intersecting with the complement. To fill just one region:

```
In : import skimage.morphology as mo
In : import scipy.ndimage as ndi
In : n = io.imread('nicework.png')/255
In : nb = n - bwerode(n,sq)
In : nb1 = nb[0:120,0:80]
In : nf = ndi.binary_fill_holes(nb1,sq)
In : nb[0:120,0:80] = nf
```

Python

The results are shown in Figure 10.27. Image (a) is the original, (b) the boundary, and (c) the result of a region fill. Figure (d) shows a variation on the region filling, we just include all boundaries. This was

obtained with

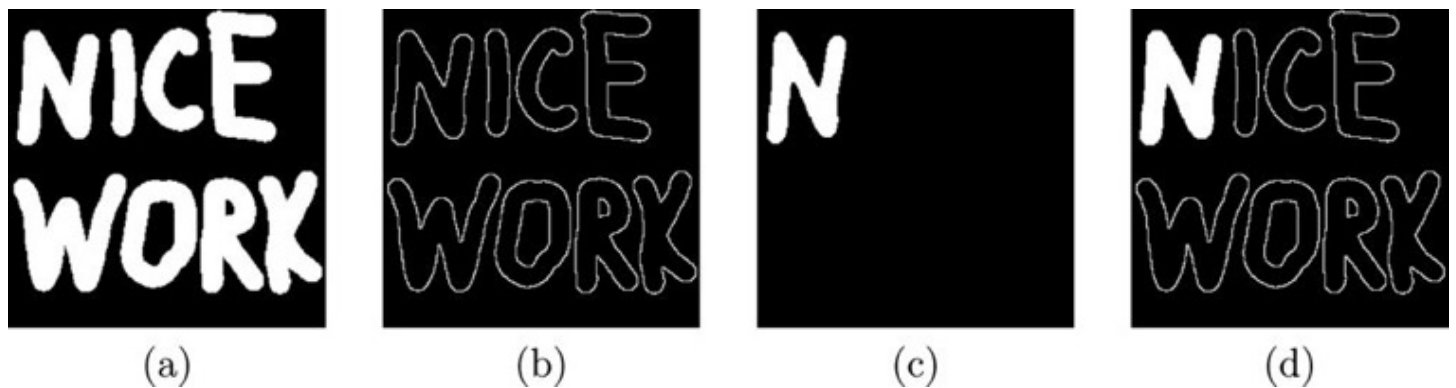
```
>> figure, imshow(nf|nb)
```

MATLAB/Octave

Skeletonization

Recall that the *skeleton* of an object can be defined by the “medial axis transform”; we may imagine fires burning in along all edges of the object. The places where the lines of fire meet form the skeleton. The skeleton may be produced by morphological methods.

Figure 10.27: Region filling



Consider the table of operations as shown in Table 10.1.

Table 10.1 Operations used to construct the skeleton

Erosions	Openings	Set differences
A	$A \circ B$	$A - (A \circ B)$
$A \ominus B$	$(A \ominus B) \circ B$	$(A \ominus B) - ((A \ominus B) \circ B)$
$A \ominus 2B$	$(A \ominus 2B) \circ B$	$(A \ominus 2B) - ((A \ominus 2B) \circ B)$
$A \ominus 3B$	$(A \ominus 3B) \circ B$	$(A \ominus 3B) - ((A \ominus 3B) \circ B)$
$\vdots$	$\vdots$	$\vdots$
$A \ominus kB$	$(A \ominus kB) \circ B$	$(A \ominus kB) - ((A \ominus kB) \circ B)$

Here we use the convention that a sequence of k erosions using the same structuring element B is denoted $A \ominus kB$. We continue the table until $(A \ominus kB) \circ B$ is empty. The skeleton is then obtained by taking the

unions of all the set differences. An example is given in Figure 10.28, using the cross-shaped structuring element.

Since $(A \ominus 2B) \circ B$ is empty, we stop here. The skeleton is the union of all the sets in the third column; it is shown in Figure 10.29. This method of skeletonization is called *Lantuéjoul's method*; for details, see Serra [45].

Programs are given at the end of the chapter. We shall experiment with the nice work image, either with MATLAB or Octave:

```
>> nk = imskel(n,sq);  
>> imshow(nk)  
>> nk2 = imskel(n,cr);  
>> figure,imshow(nk2)
```

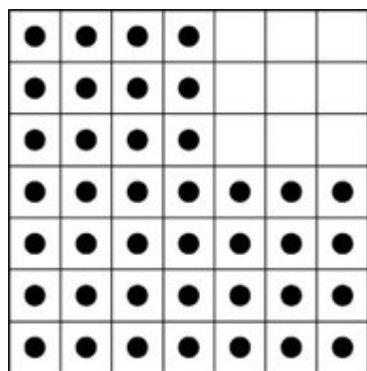
MATLAB/Octave

or with Python:

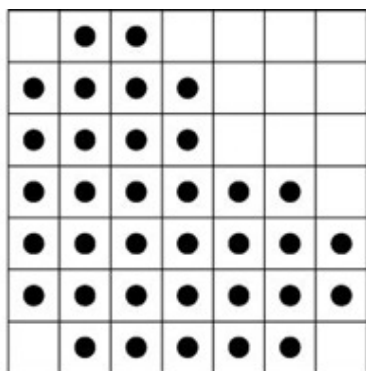
```
In : nk = bwskel(n,sq)  
In : io.imshow(nk)  
In : nk2 = bwskel(n,cr)  
In : f = plt.figure(); f.show(io.imshow(nk2))
```

Python

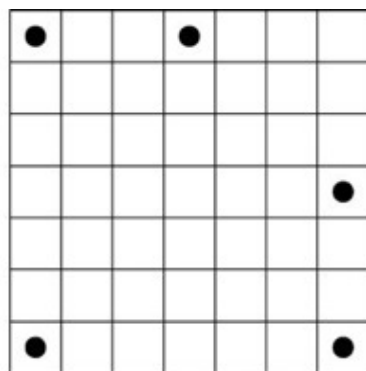
Figure 10.28: Skeletonization



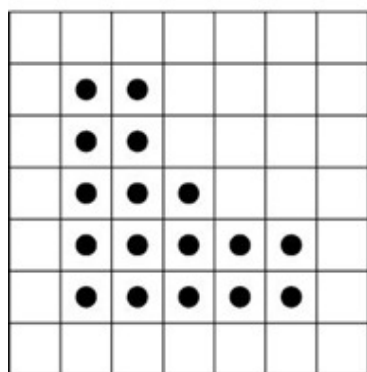
A



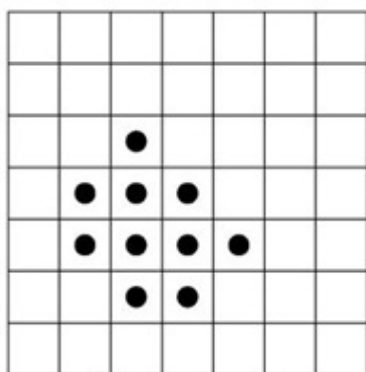
$A \circ B$



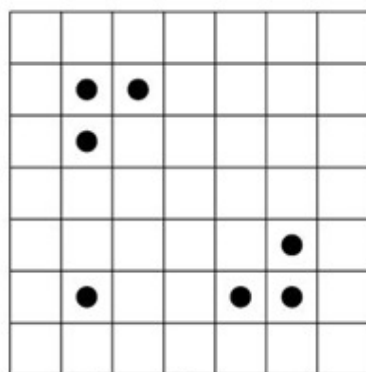
$A - (A \circ B)$



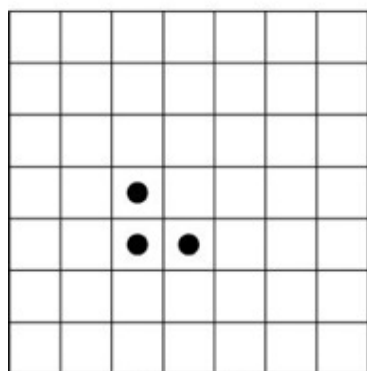
$A \ominus B$



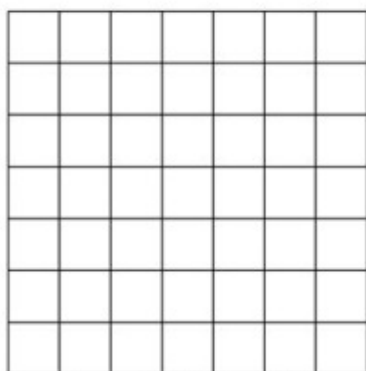
$(A \ominus B) \circ B$



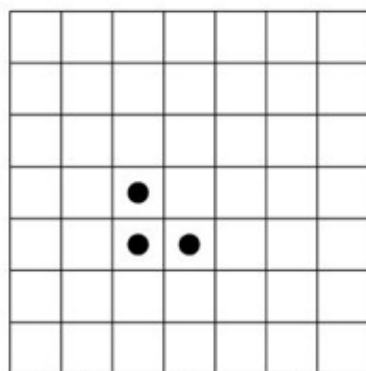
$(A \ominus B) - ((A \ominus B) \circ B)$



$A \ominus 2B$

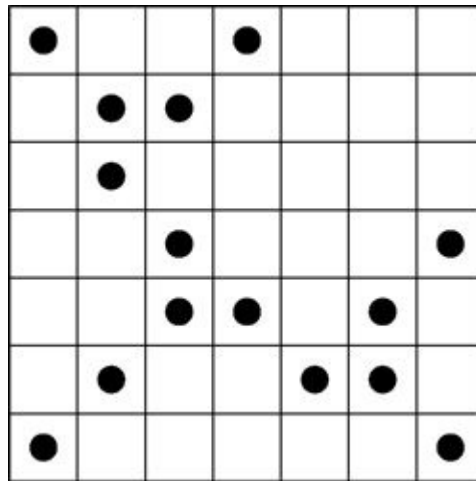


$(A \ominus 2B) \circ B$



$(A \ominus 2B) - ((A \ominus 2B) \circ B)$

Figure 10.29: The final skeleton



The results are shown in Figure 10.30. Image (a) is the result using the square structuring element; image (b) is the result using the cross structuring element.

Figure 10.30: Skeletonization of a binary image



(a)



(b)

10.7 A Note on the `bwmorph` Function in MATLAB and Octave

The theory of morphology developed so far uses versions of erosion and dilation sometimes called *generalized* erosion and dilation; so called because the definitions allow for general structuring elements. This is the method used in the `imerode` and `imdilate` functions. However, the `bwmorph` function actually uses a different approach to morphology; that based on *lookup tables*. (Note however that Octave's `bwmorph` function implements erosion and dilation with calls to the `imerode` and `imdilate` functions.) We have seen the use of lookup tables for binary operations in Chapter 11; they can be just as easily applied to implement morphological algorithms.

The idea is simple. Consider the 3×3 neighborhood of a pixel. Since each pixel in the neighborhood can only have two values, there are $2^9 = 512$ different possible neighborhoods. We define a morphological operation to be a function that maps these neighborhoods to the values 0 and 1. Each possible neighborhood state can be associated with a numeric value from 0 (all pixels have value 0) to 511 (all

pixels have value 1). The lookup table is then a binary vector of length 512; its k -th element is the value of the function for state k .

With this approach, we can define dilation as follows: a 0-valued pixel is changed to 1 if at least one of its eight neighbors has value 1. Conversely, we may define erosion as changing a 1-valued pixel to 0 if at least one of its eight neighbors has value 0.

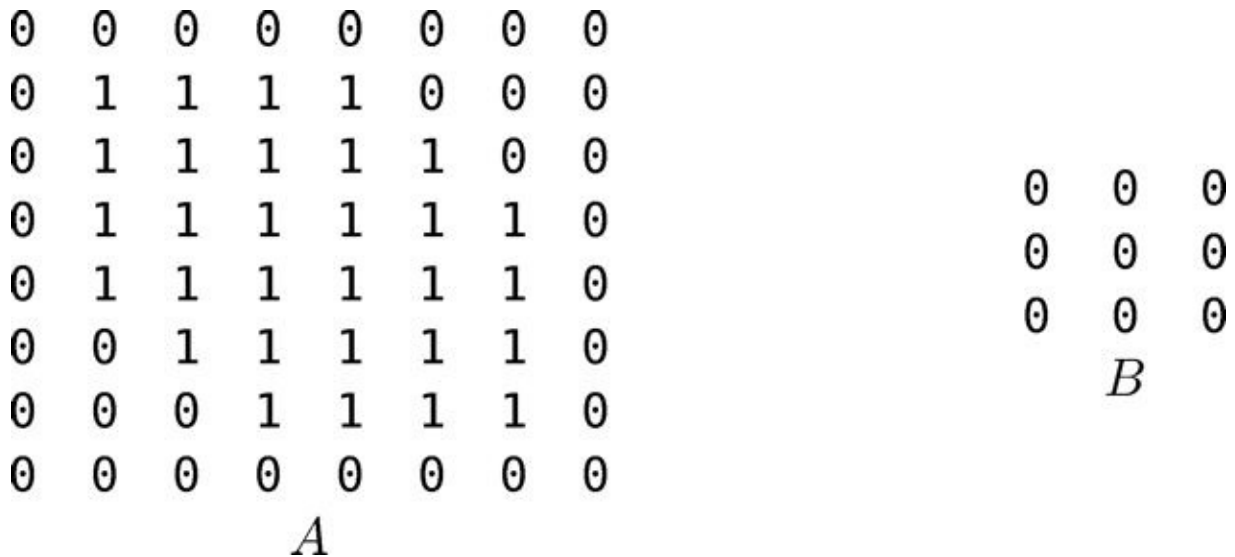
Many other operations can be defined by this method (see the help file for `bwmorph`). The advantage is that any operation can be implemented extremely easily, simply by listing the lookup table. Moreover, the use of a lookup table allows us to satisfy certain requirements; the skeleton, for example, can be connected and have exactly one pixel thickness. This is not necessarily true of the algorithm presented in Section 10.6. The disadvantage of lookup tables is that we are restricted to using 3×3 neighborhoods. For more details, see “Digital Image Processing” by W. K. Pratt [35].

10.8 Grayscale Morphology

The operations of erosion and dilation can be generalized to be applied to grayscale images. But before we do, we shall reconsider binary erosion and dilation. We have defined binary erosion, $A \ominus B$, to be the union of the $(0, 0)$ positions of all translations B_x for which $B_x \subseteq A$.

Suppose we take B to be a 3×3 square consisting entirely of zeros. Let A be the image as shown in Figure 10.31. Now suppose we move over the image A , and for each point p we perform the following steps:

Figure 10.31: An example for erosion



1. Find the 3×3 neighborhood N_p of p
2. Compute the matrix $N_p - B$
3. Find the minimum of that result.

We note that since B consists of all zeros, the second and third items could be reduced to finding the minimum of N_p . However, we shall see that for generalization it will be more convenient to have this expanded form. An immediate consequence of these steps is that if a neighborhood contains at least one zero, the output will be zero. The output is one only if the neighborhood contains all ones. For example:

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{array} \longrightarrow 0$$

If we perform this operation, we will obtain:

$$\begin{array}{cccccc} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

which you can verify is exactly the erosion $A \ominus B$.

For dilation, we perform a sequence of steps very similar to those for erosion:

1. Find the 3×3 neighborhood N_p of p
2. Compute the matrix $N_p + B$
3. Find the maximum of that result.

We note again that since B consists of all zeros, the second and third items could be reduced to finding the maximum of N_p . If the neighborhood contains at least one 1, then the output will be 1. The output will be 0 only if the neighborhood contains all zeros.

Suppose A to be surrounded by zeros, so that neighborhoods are defined for all points in A above. Applying these steps produces:

$$\begin{array}{cccccc} 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \end{array}$$

which again can be verified to be the dilation $A \oplus B$.

If A is a grayscale image, and B is a structuring element, which will be an array of integers, we define grayscale erosion by using the steps above; for each pixel p in the image:

1. Position B so that $(0, 0)$ lies over p
2. Find the neighborhood N_p of p corresponding to the shape of B
3. Find the value $\min(N_p - B)$.

We note that there is nothing in this definition which requires B to be any particular shape or size or that the elements of B be positive. And as for binary dilation, B does not have to contain the origin $(0, 0)$.

We can define this more formally; let B be a set of points with associated values. For example, for our square of zeros we would have:

Point	Value
$(-1, -1)$	0
$(-1, 0)$	0
$(-1, 1)$	0
$(0, -1)$	0
$(0, 0)$	0
$(0, 1)$	0
$(1, -1)$	0
$(1, 0)$	0
$(1, 1)$	0

The set of points forming B is called the domain of B and is denoted D_B . Now we can define:

$$(A \ominus B)(x, y) = \min\{A(x + s, y + t) - B(s, t), (s, t) \in D_B\},$$

$$(A \oplus B)(x, y) = \max\{A(x + s, y + t) + B(s, t), (s, t) \in D_B\}.$$

We note that some published definitions use $s - x$ and $t - y$ instead of $x + s$ and $y + t$. This just requires that the structuring element is rotated 180°.

Figure 10.32: An example for grayscale erosion and dilation

		y							t		
		1	2	3	4	5	s		-1	0	1
x	1	10	20	20	20	30			1	2	3
	2	20	30	30	40	50			4	5	6
	3	20	30	30	50	60			7	8	9
	4	20	40	50	50	60					
	5	30	50	60	60	70					

A

B

An example. Suppose we take A and B as given in Figure 10.32.

In this example $(0,0) \in D_B$. Consider $(A \ominus B)(1, 1)$. By the definition, we have:

$$(A \ominus B)(1, 1) = \min\{A(1 + s, 1 + t) - B(s, t), (s, t) \in D_B\}.$$

Since $D_B = \{(s, t): -1 \leq s \leq 1; -1 \leq t \leq 1\}$, we have

$$(A \ominus B)(1, 1) = \min\{A(1 + s, 1 + t) - B(s, t) : -1 \leq s \leq 1; -1 \leq t \leq 1\}.$$

In order to ensure we do not require matrix indices that move outside A , we simply “cut off” the structuring element so that we restrict it to elements in A .

The values of $A(1 + s, 1 + t) - B(s, t)$ can then be obtained by matrix arithmetic:

$$\begin{bmatrix} 10 & 20 \\ 20 & 30 \end{bmatrix} - \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix} = \begin{bmatrix} 5 & 14 \\ 12 & 21 \end{bmatrix}$$

and the minimum of the result is 5. Note that to create the matrices we ensure that the $(0,0)$ point of B sits over the current point of A , in this case the point $(1,1)$. Another example:

$$(A \ominus B)(3, 4) = \min \left\{ \begin{bmatrix} 30 & 40 & 50 \\ 30 & 50 & 60 \\ 50 & 50 & 60 \end{bmatrix} - \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \right\} = \min \left\{ \begin{bmatrix} 29 & 38 & 47 \\ 26 & 45 & 54 \\ 43 & 42 & 51 \end{bmatrix} \right\} = 26.$$

Finally, we have

$$A \ominus B = \begin{bmatrix} 5 & 6 & 14 & 15 & 16 \\ 8 & 9 & 17 & 18 & 19 \\ 12 & 13 & 25 & 26 & 39 \\ 15 & 16 & 28 & 29 & 46 \\ 18 & 19 & 39 & 48 & 49 \end{bmatrix}$$

Dilation is very similar to erosion, except that we add B and take the maximum of the result. As for erosion we restrict the structuring element so that it does not go outside of A . For example:

$$(A \oplus B)(1, 1) = \max \left\{ \begin{bmatrix} 10 & 20 \\ 20 & 30 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix} \right\} = \max \left\{ \begin{bmatrix} 15 & 26 \\ 28 & 39 \end{bmatrix} \right\} = 39,$$

$$(A \oplus B)(3, 4) = \max \left\{ \begin{bmatrix} 30 & 40 & 50 \\ 30 & 50 & 60 \\ 50 & 50 & 60 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \right\} = \max \left\{ \begin{bmatrix} 31 & 42 & 53 \\ 34 & 55 & 66 \\ 57 & 58 & 69 \end{bmatrix} \right\} = 69.$$

After all the calculations, we have

$$A \oplus B = \begin{bmatrix} 39 & 39 & 49 & 59 & 58 \\ 39 & 39 & 59 & 69 & 68 \\ 49 & 59 & 59 & 69 & 68 \\ 59 & 69 & 69 & 79 & 78 \\ 56 & 66 & 66 & 76 & 75 \end{bmatrix}$$

Note that an erosion may contain negative values, or a dilation value greater than 255. In order to render the result suitable for display, we have the same choices as for spatial filtering: we may apply a linear transformation or we may clip values.

In general, and this can be seen from the examples, erosion will tend to decrease and darken objects in an image, and dilation will tend to enlarge and lighten objects.

Relationship between Grayscale Erosion and Dilation

By definition of maximum and minimum we have, if X and Y are two matrices:

$$\max\{X + Y\} = -\min\{-X - Y\}.$$

Since $\max\{X + Y\}$ corresponds to $A \oplus B$ and $\min\{X - Y\}$ to $A \ominus B$, we have

$$\begin{aligned} A \oplus B &= -(-A \ominus B), \\ A \ominus B &= -(-A \oplus B), \end{aligned}$$

or

$$\begin{aligned} -(A \oplus B) &= -A \ominus B, \\ -(A \ominus B) &= -A \oplus B. \end{aligned}$$

We can use the `imerode` and `imdilate` functions for grayscale erosion and dilation, but we have to be more careful about the structuring element. To create a structuring element for use with grayscale morphology, we can either use a binary mask, or we have to provide both a neighborhood D_B , and its values. To do this we need to use the `strel` function.

For example, we shall use MATLAB with the previous examples. First we need to create the structuring element.

```
>> str = strel('arbitrary',ones(3,3),[1 2 3;4 5 6;7 8 9])
```

```
str =
```

```
Nonflat STREL object containing 9 neighbors.
```

```
Neighborhood:
```

1	1	1
1	1	1
1	1	1

```
Height:
```

1	2	3
4	5	6
7	8	9

MATLAB/Octave

Here we use the `arbitrary` parameter of `strel`; this allows us to create a structuring element containing any values we like. The first matrix: `ones(3,3)` provides the neighborhood; the second matrix provides the values. Now we can test it.

```
>> A = [10 20 20 20 30;20 30 30 40 50;20 30 30 50 60;20 40 50 50 60;30 50  
60 60 70];  
>> imerode(A,str)
```

```
ans =
```

5	6	14	15	16
8	9	17	18	19
12	13	25	26	39
15	16	28	29	46
18	19	39	48	49

MATLAB/Octave

Python provides this generalized erosion as `gray_erosion` in the `ndimage` module of `scipy`:

```
In : A = np.array([[10,20,20,20,30],[20,30,30,40,50],[20,30,30,50,60],  
[20,40,50,50,60],[30,50,60,60,70]]).astype('uint8')  
In : struct = np.array([[1,2,3],[4,5,6],[7,8,9]])  
In : print ndi.gray_erosion(A,footprint=mo.square(3),structure=struct)  
[[ 5  6 14 15 16]  
 [ 8  9 17 18 19]  
[12 13 25 26 39]  
[15 16 28 29 46]  
[18 19 39 48 49]]
```

Python

For dilation MATLAB implements the convention of the structuring element being rotated by 180°. So to obtain the result we produced before, we need to rotate `str` to obtain `str2`.

```
>> str2 = strel('arbitrary',ones(3,3),[9 8 7;6 5 4;3 2 1])
```

```
str2 =
```

```
Nonflat STREL object containing 9 neighbors.
```

```
Neighborhood:
```

```
  1    1    1
  1    1    1
  1    1    1
```

```
Height:
```

```
  9    8    7
  6    5    4
  3    2    1
```

```
>> imdilate(A,str2)
```

```
ans =
```

```
 39    39    49    59    58
 39    39    59    69    68
 49    59    59    69    68
 59    69    69    79    78
 56    66    66    76    75
```

MATLAB/Octave

In Python the same result is obtained with `gray_dilation`:

```
In : struct2 = np.array([[9,8,7],[6,5,4],[3,2,1]])
In : print ndi.gray_dilation(A,footprint=mo.square(3),\
...: structure=struct,mode='constant')
[[39 39 49 59 59]
 [39 39 59 69 69]
 [49 59 59 69 69]
 [59 69 69 79 79]
 [59 69 69 79 79]]
```

Python

Now we can experiment with an image. We would expect that dilation would increase light areas in an image, and erosion would decrease them.

```
>> c = imread('caribou.png');
>> str = strel('square',5);
>> cd = imdilate(c,str);
>> ce = imerode(c,str);
>> imshow(cd), figure,imshow(ce)
```

MATLAB/Octave

Using Python:

```
In : c = io.imread('caribou.png')
In : str = mo.square(5)
In : cd = mo.dilation(c,str)
In : ce = mo.erosion(c,str)
In : io.imshow(cd)
In : f = plt.figure(); f.show(io.imshow(ce))
```

Python

The results are shown in Figure 10.33. Image (a) is the dilation and image (b) the erosion.

Figure 10.33: Grayscale dilation and erosion



(a)



(b)

Opening and Closing

Opening and closing are defined exactly as for binary morphology: opening is an erosion followed by a dilation, and closing is a dilation followed by an erosion. The `imopen` and `imclose` functions can be applied to grayscale images, if a structuring element has been created with `strel`.

Using the caribou image and the same 5×5 square structuring element as above:

```
>> co = imopen(c,str);
>> cc = imclose(c,str);
>> imshow(co),figure,imshow(cc)
```

MATLAB/Octave

and the results are shown in Figure 10.34; image (a) being the opening and (b) the closing.

10.9 Applications of Grayscale Morphology

Almost all the applications we saw for binary images can be carried over directly to grayscale images.

Figure 10.34: Grayscale opening and closing



(a)



(b)

Edge Detection

We can use the morphological gradient

$$(A \oplus B) - (A \ominus B)$$

for grayscale edge detection. We can try this with two different structuring elements.

```
>> str1 = strel('disk',1);
>> str2 = strel('square',5);
>> imshow(imdilate(x,str1)-imerode(c,str1))
```

MATLAB/Octave

or

```
In : str1 = mo.disk(1)
In : str2 = mo.square(5)
In : io.imshow(mo.dilation(c,str1)-mo.erosion(c,str1))
```

Python

and similarly for `str2`. The results are shown in Figure 10.35. Image (a) uses the 3×3 cross, and image (b) the 5×5 square. Another edge detection method [46, 47] applied to an image A with structuring element B is defined by

$$\min\{(A \oplus B) - A, A - (A \ominus B)\}. \quad (10.1)$$

An example using the $s \times 3$ square structuring element is given in Figure 10.36(a). Another edge filter, which works well in the presence of noise, first blurs the image using the *alpha-trimmed mean filter*. This non-linear filter first orders the elements in the neighborhood, trims off a fraction α of the lowest and highest values, and returns the mean of the rest. This filter can be implemented (with $\alpha = 0.25$) as follows:

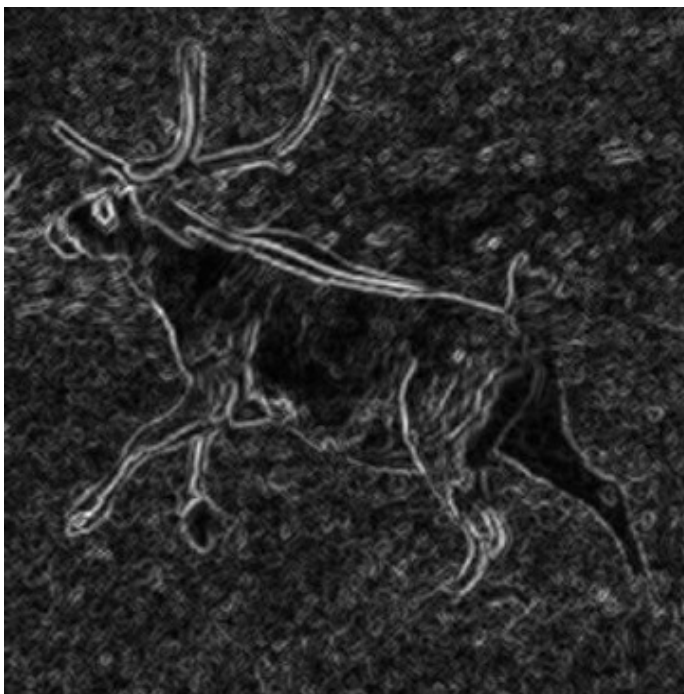
```
function out = atmean(r)
    s = sort(r(:));
    d = floor(length(s)/4);
    out = mean(s(1+d:end-d));
end
```

MATLAB/Octave

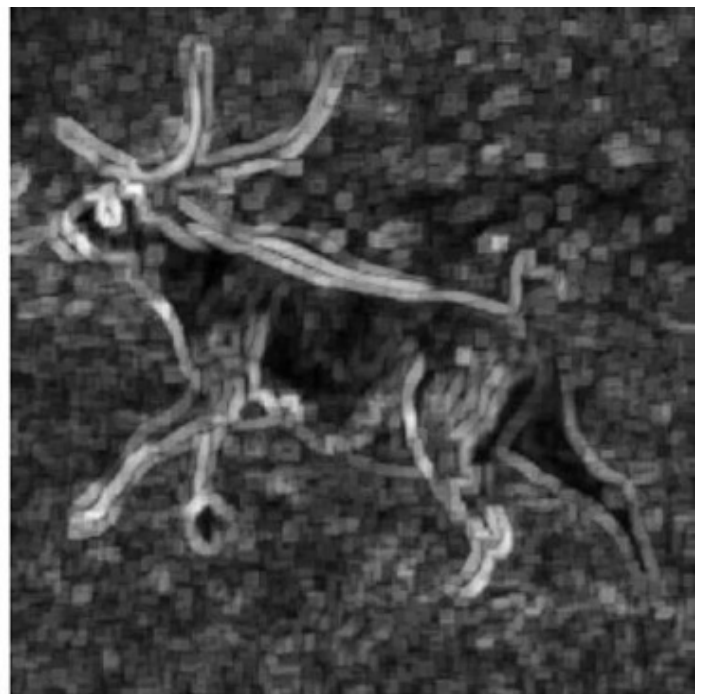
```
def atmean(r):
    s = np.sort(r.flatten())
    d = len(s)/4
    return s[d:-d].mean()
```

Python

Figure 10.35: Use of the morphological gradient



(a)



(b)

and then applied to the image with either


```
>> ca = nlfilter(c,[3,3],@atmean)
```

MATLAB/Octave

or

```
In : ca = ndi.generic_filter(c,atmean,size=(3,3))
```

Python

Given the result T of the filter, the edge filter is defined as

$$\min\{(T \circ B) - (T \ominus B), (T \oplus B) - (T \bullet B)\}. \quad (10.2)$$

and an example is shown in Figure 10.36(b).

Figure 10.36: Morphological edge finding



(a) Using Equation 10.1



(b) Using Equation 10.2

Noise Removal

As before we can attempt to remove noise with morphological filtering: an opening followed by a closing. We can apply the following commands with `str` being either the 3×3 square or the cross:

```
>> cn = imnoise(c,'salt_&_pepper');  
>> cf = imclose(imopen(cn,str),str);  
>> imshow(cn),figure,imshow(cf)
```

MATLAB/Octave

or

```

In : cn = ut.noise.random_noise(c,'s&p')
In : cn = ut.img_as_ubyte(cn)
In : cno = mo.opening(cn,str)
In : cnoc = mo.closing(cno,str)
In : io.imshow(cn)
In : f = plt.figure(); f.show(io.imshow(cnoc))

```

Python

These are shown in Figure 10.37. The result is reasonable, if slightly blurry. Morphological filtering does not perform particularly well on Gaussian noise.

Figure 10.37: Use of morphological filtering to remove noise



(a) Salt and pepper noise



(b) Cleaning with 3×3 square



(c) Cleaning with cross

10.10 Programs

Here are programs for skeletonization by Lantuéjoul's method, first in MATLAB or Octave:

```

function skel = imskel(image, kernel)
% IMSKEL - Calculates the skeleton of IMAGE using kernel KERNEL
%
skel=zeros(size(image));
e=image;
while (any(e(:))),
    o=imopen(e,kernel);
    skel=skel | (e&~o);
    e=imerode(e,kernel);
end

```

MATLAB/Octave

and in Python:

```
def bwskel(im,ker):
    skel = np.zeros_like(im)
    from skimage.morphology import binary_erosion, binary_opening
    e = (np.copy(im)>0)*1
    while e.max()>0:
        o = binary_opening(e,ker)
        skel = skel | (e & 1-o)
        e = binary_erosion(e,ker)
    return skel
```

Python

Exercises

- For each of the following images A_i and structuring elements B_j :

$A_1 =$

```
0 0 0 0 0 0 0 0
0 0 0 1 1 1 1 0
0 0 0 1 1 1 1 0
0 1 1 1 1 1 1 0
0 1 1 1 1 1 1 0
0 1 1 1 1 0 0 0
0 1 1 1 1 0 0 0
0 0 0 0 0 0 0 0
```

$A_2 =$

```
0 0 0 0 0 0 0 0
0 1 1 1 1 1 1 0
0 1 1 1 1 1 1 0
0 1 1 0 0 1 1 0
0 1 1 0 0 1 1 0
0 1 1 1 1 1 1 0
0 1 1 1 1 1 1 0
0 0 0 0 0 0 0 0
```

$A_3 =$

```
0 0 0 0 0 0 0 0
0 0 0 0 0 1 1 0
0 1 1 1 0 1 1 0
0 1 1 1 0 1 1 0
0 1 1 1 0 1 1 0
0 1 1 1 0 0 0 0
0 1 1 1 0 0 0 0
0 0 0 0 0 0 0 0
```

$B_1 =$

```
0 1 0
1 1 1
0 1 0
```

$B_2 =$

```
1 1 1
1 1 1
1 1 1
```

$B_3 =$

```
1 0 0
0 0 0
0 0 1
```

calculate the erosion $A_i \ominus B_j$, the dilation $A_i \oplus B_j$, the opening $A_i \circ B_j$, and the closing $A_i \bullet B_j$. Check your answers with your computer system.

- Suppose a square object was eroded by a circle whose radius was about one quarter the side of the square. Draw the result.
- Repeat the previous question with dilation.
- Using the binary images `circles.png`, `circles2.png`, `meet_text.png`, `nicework.png`, and `rings.png`, view the erosion and dilation with both the square and the cross-shaped structuring elements. Can you see any differences?
- Read in the image `circles2.png`.

- (a) Erode with squares of increasing size until the image starts to split into disconnected components.
- (b) Find the coordinates of a pixel in one of the components.
- (c) Use the appropriate commands to isolate that particular component.

- 6.
 - (a) With your disconnected image from the previous question, compute its boundary.
 - (b) Again find a pixel inside one of the boundaries.
 - (c) Use the region filling function to fill that region.
 - (d) Display the image as a boundary with one of the regions filled in.
- 7. Using the 3×3 square structuring element, compute the skeletons of
 - (a) A 7 square
 - (b) A 5×9 rectangle
 - (c) An L shaped figure formed from an 8×8 square with a 3×3 square taken from a corner
 - (d) An H shaped figure formed from a 15×15 square with 5×5 squares taken from the centers of the top and bottom
 - (e) A cross formed from an 11×11 square with 3×3 squares taken from each corner

In each case, check your answer with your computer system.

- 8. Repeat the above question but use the cross-shaped structuring element.
- 9. For the images listed in Question 4, obtain their skeletons by both supplied functions, and by using Lantuéjoul's method. Which seems to provide the best result?
- 10. Use the hit-or-miss transform with appropriate structuring elements to find the dot on the "i" in the word "Friday" in the image `meet_text.png`.
- 11. Let A be the matrix:

35	1	6	26	19	24
3	32	7	21	23	25
31	9	2	22	27	20
8	28	33	17	10	15
30	5	34	12	14	16
4	36	29	13	18	11

(If you are using MATLAB or Octave, this matrix can be obtained with `A=magic(6)`. If you are using Python you will have to enter this matrix manually.) By hand, compute the grayscale erosions and dilations with the structuring elements $B =$

10	10	10	5	20	5
10	10	10	20	5	20
10	10	10	5	20	5

In each case, check your answers with your computer system.

- 12. Perform grayscale erosions, dilations, openings, and closings on the cameraman image using the above structuring elements.
- 13.
 - (a) Obtain the grayscale version of the twins image as was used in Chapter 8.
 - (b) Apply salt and pepper noise to the image.
 - (c) Attempt to remove the noise using morphological techniques.
 - (d) Compare the results to median filtering.
- 14. Repeat the above question but use Gaussian noise.
- 15. Use morphological methods to find the edges of the image `stairs.png`. Do this in two ways:
 - (a) By thresholding first and using binary morphology.
 - (b) By using grayscale morphology and thresholding second.

Which seems to give the best results?

- 16. Compare the results of the previous question with standard edge detection techniques.

11 Image Topology

11.1 Introduction

We are often interested in only the very basic aspects of an image: the number of occurrences of a particular object; whether there are holes, and so on. The investigation of these fundamental properties of an image is called *digital topology* or *image topology* and in this chapter we shall investigate some of the more elementary aspects of this subject.

For example, consider an image thresholded and cleaned up with a morphological opening to show a collection of blobs. Such an image can be obtained with MATLAB or Octave by:

```
>> p = imread('pinenuts.png');  
>> p2 = ~im2bw(p,graythresh(p));  
>> p3 = imopen(p2,strel('disk',5));
```

MATLAB/Octave

or with Python by:

```
In : import skimage.morphology as mo  
In : p = io.imread('pinenuts.png')  
In : p2 = 1.0-(p>fl.threshold_otsu(p))  
In : p3 = mo.opening(p2,mo.disk(5))
```

Python

The image `n2` is shown in Figure 11.1. The number of blobs can be determined by morphological